

VLA_Arch_Spec_v1

DEA-8 视觉-语言-动作加速器架构与 RTL 实现规范

π_0 Diffusion Expert 专用推理加速器

1 概述与范围

1.1 文档目的

本文档定义 DEA-8 加速器的架构、微架构与 RTL 实现契约。DEA-8 是面向 π_0 视觉-语言-动作模型 (Vision-Language-Action, VLA) 中 Diffusion Expert (动作专家) 子网络的专用推理加速器。

π_0 的推理分为两个阶段：视觉-语言模型 (PaliGemma 2B) 对观测与指令做一次 prefill，产生 prefix KV cache；随后动作专家以流匹配 (flow matching) 方式迭代 10 个去噪步，生成一个包含 50 个时间步的动作块。本文档只覆盖后者。



- 灰色高亮部分属于未确认，待调整内容

1.2 范围

1.2.1 范围内

- MXINT8-B16 权重与激活在 8 个 16×16 INT8 矩阵核上的执行。
- BF16 持久高精度状态配 FP32 数值内核。
- 分组查询注意力 (GQA)，8 个 query head、1 个共享 KV head。
- 880 ($\text{ceil}(867/16) * 16$) 个物理 key 槽位上的在线 softmax，其中至多 867 个是逻辑有效的。
- 残差、RMSNorm、RoPE、GELU、time embedding、小投影与 Euler 更新。
- 静态描述符执行、全局计算停顿语义、SRAM 分体、HBM 流式取数与错误上报。
- 复位/上电、时钟门控、DFT、ECC 与故障隔离。

1.2.2 范围外

- Prefix 视觉-语言编码器的执行与 prefix KV 的生成。

- 训练、梯度计算、优化器状态、稀疏专家路由或动态批处理。
- 原生 OCP k=32 互换。片外产物一律使用本项目的 k=16 profile。

1.3 设计指标摘要

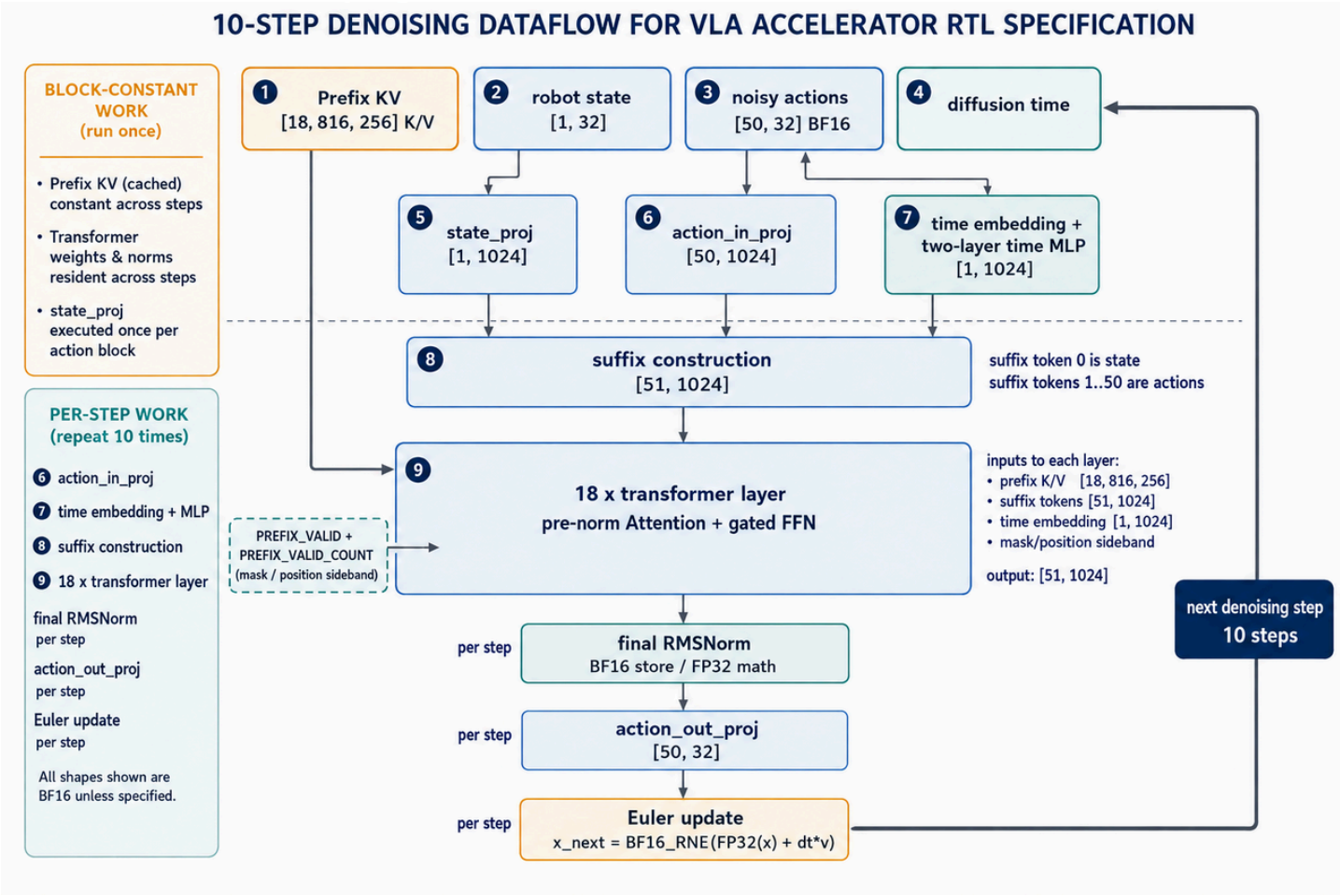
项目	指标	备注
计算阵列	8 PCore × 16×16 INT8 乘加树 = 2,048 MAC	4.096 TOPS @ 1 GHz
数值格式	MXINT8-B16 (量化块 = 计算tile = 加法树= FA B_kv)	
高精度域	BF16 存储 + FP32 计算	§ 4.1
主存	HBM3, 9 个 pseudo-channel	8 个 PCore 私有 + 1 个 GCore 共享
片上存储	GCore ≈ 1.15 MiB + 8 × PCore 160 KiB (逻辑占用 128 KiB) + EHU/CCU 0.32 MB ≈ 2.72 MB	§ 10.1, § 10.2

1.4 模型兼容

参数	基线	架构结果
隐藏宽度 D	1,024	64 个 K-tile；残差行 2,048 BF16 字节
MLP 宽度 F	4,096	每 PCore 512 (4096/8) 个输出通道，gate/up 各 32 (512/16) 个 N-tile
层数 L	18	5 bit 层计数器
Query head / KV head	8 / 1	每 PCore 一个 query head；KV 共享并广播 (MQA)
Head 维 d_h	256	每 head 16 个特征块 (256/16)
Prefix 容量	816	51 个物理 k=16 块；基线全有效，有效性由 PREFIX_VALID 描述
Suffix 长度	51	1 个 state token + 50 个 action token
物理注意力宽度	880	55 个块；末 13 个槽位为 padding，恒被掩蔽
去噪步数	10	4 bit 步计数器，每步一份固定程序

动作视界 / 维度	[51, 32]	action_in_proj 与 action_out_proj
RMSNorm 形式	无均值扣除、无 bias、增益写作 $(1 + \gamma)$	γ 离线折进后继权重，见 § 8.11
RMSNorm ϵ	由兼容性清单给出，逐层可不同	FP32 正标量
FFN 激活	GELU(tanh 近似) $\odot$	§ 4.11 给出误差界

1.5 端到端功能数据流



设 s 为 32 元素机器人状态， x_t 为第 t 个去噪步的 50×32 含噪动作张量， τ_t 为标量扩散时刻。Suffix 隐状态 h_t^0 有 51 行：第 0 行是投影后的 state token，第 1–50 行是条件化后的 action token。Prefix K/V 由 prefix 引擎逐层提供，在整个动作块期间只读。加速器在十次 Euler 更新后产出 x_0 。

代码块

```
1  h_state = state_proj(s) # [1,1024]
2  a0      = action_in_proj(x_t) # [50,1024]
3  e_t     = time_embedding(tau_t) # [1,2048] ← 离线 LUT
4  c_t     = action_time_mlp_out(phi(action_time_mlp_in(e_t)))
5  h_action[m] = a0[m] + c_t # [50,1024]
```

```

6  h_t^0    = concat(h_state, h_action[0:49])          # [51,1024]
7
8  h_t^18   = Transformer18(h_t^0, prefix_KV, valid_mask)
9  y_t      = final_rmsnorm(h_t^18[1:50])             # [50,1024]
10
11 v_t       = action_out_proj(y_t)                   # [50,32]
12 x_(t+1) = BF16_RNE( FP32(x_t) + dt_t * FP32(v_t) )

```

1.6 单层 Transformer 方程

对第 ℓ 层， R_ℓ 是入口的 BF16 残差张量 $[51, D]$ 。下列表达式定义**架构上的数学结果**；其分块执行由 § 8 定义。**y 向量离线折进后继权重矩阵**。除非兼容性清单显式开启，否则不存在 bias 项。

代码块

```

1  N_a    = R_l * rsqrt( (1/D) * sum_k R_l[:,k]^2 + eps_a[l] )
2  Q_h    = N_a * W_q[h]                                for h = 0..7      # [51,256]
3  K_s,V_s = N_a * W_kv                                  # [51,256] each
4  Q_h,K_s = RoPE(Q_h, K_s, suffix_position)
5  S_h[m,j] = dot(Q_h[m,:], K_ell[j,:]) / sqrt(d_h)
6  A_h[m,j] = mask_softmax_j(S_h[m,j], valid(m,j))
7  C_h    = A_h * V_ell                                  # [51,256]
8  R_mid = BF16_RNE( R_ell + concat_h(C_h) * W_o )      # [51,1024]
9  N_f    = R_mid * rsqrt( (1/D) * sum_k R_mid[:,k]^2 + eps_f[ell] )
10 G      = N_f * W_gate ; U = N_f * W_up               # [51,4096]
11 Z      = GELU(G) elementwise_mul U
12 R_(ell+1) = BF16_RNE( R_mid + Z * W_down )           # [51,1024]2 符号、
               术语与缩写

```

2 符号、术语与缩写

2.1 模型侧符号

张量索引一律从 0 开始，闭区间用 $[a, b]$ ，半开区间用 $[a, b)$ 。

符号	定义	值 / 范围
D, d_{model}	Transformer 隐藏宽度	1,024
F, d_{ff}	门控 FFN 中间宽度	4,096
H_q / H_{kv}	Query / KV head 数	8 / 1 (MQA)

d_h	每 head 特征宽度	256
L	层数	18
M, T_s	Suffix 长度（流式维）	51
P, T_p	Prefix 容量	816
S	逻辑序列上限 = P + M	867
T_phys	padding 后的物理注意力宽度	880
N_block	KV 块数 = T_phys / 16	55
m	Suffix query / token 索引	$0 \leq m < 51$
j	物理 key 索引	$0 \leq j < 880$
jj	Suffix 内列号 = j - 816	$0 \leq jj < 64$ （含 padding）
b	KV 块索引	$0 \leq b < 55$
r	token 内元素索引	$0 \leq r < D$ 或 d_h
t, k	去噪步索引	$0 \leq t < 10$
ℓ	Transformer 层索引	$0 \leq \ell < 18$
ϵ	RMSNorm epsilon	逐层 FP32 正标量
dt_t	第 t 步的 Euler 步长	FP32 标量；基线 $\Delta t = 1/10$
$\mu[m], \ell[m]$	在线 softmax 的行最大值与归一化和	FP32
$\alpha_b[m]$	第 b 块的重缩放因子	FP32

2.2 硬件侧符号

符号	定义	值
TILE, TILE_R, TILE_C	阵列行数 / 列数 = K-tile 粒度 = MX 块大小	16
N_CORE	PCore 数 = query head 数	8
Kt, Nt	一条 GEMM 描述符的 K / N tile 数	见 § 9.7
B_kv	FlashAttention 的 KV 分块	16 (= TILE_R)
E_a, E_w, E_b	激活 / 权重 / 通用 stationary ScaleScaleblock scale (E8M0)	[0, 254]

2.3 模块缩写

缩写	全称	职责
GCore	Global Core	共享残差、RMS/量化、prefix/suffix KV、归约、EHU
PCore	Private Core	一个 query head 与一路私有权重流
MXU	MatriX Unit	16×16 INT8 乘加树，256 MAC/拍
DEQACC	DEQuantize-ACCumulate	反量化并累加，
FACC	Fp32 ACCumulator	通用 GEMM 累加器存储，乒乓
OACC	Output ACCumulator	Attention 输出累加器存储 [51,256] FP32
VPU	Vector Processing Unit	16 lane FP32 向量运算与量化
SFU	Special Function Unit	exp / reciprocal / rsqrt / GELU / sin / cos
TSQ	Tile Sequencer	PCore 内的描述符展开与地址生成
CCU	Central Control Unit	描述符发射、屏障、错误处理
MCU	Memory Control Unit	私有 HBM 权重取址与 tile 组装（8 份）
PRU	Partial Reduction Unit	GCore 内的归约汇聚、残差加、平方和
QUANT	Quantizer	GCore 的 16 lane MXINT8-B16 编码器
EHU	Elementwise / Housekeeping Unit	time embedding、Euler 更新、小投影
CNET	Collective NETwork	$8 \rightarrow 1$ FP32 归约与 $8 \rightarrow 1$ gather
XBC	X BroadCast	GCore $\rightarrow$ 8 PCore 的激活/KV 广播
HIF	Host InterFace	CSR / 命令队列 / 中断

2.4 存储器缩写

缩写	位置	内容
RES-Memory	GCore	BF16 残差流 h[51,1024]

XHAT	GCore	归一化并量化后的激活 $X \llcorner \text{INT8}$)
E_a	GCore	XHAT 的ScaleScale表 [51,64]
KV-Memory	GCore	WIN_A / WIN_B (prefix 双窗) + SUFFIX 库
Q-Memory	PCore	$Q \llcorner [1,256]$, A_FIN 后复用为 $\hat{O}$
SBUF	PCore	分数缓冲 $S[51,16]$ FP32, 乒乓
PBUF	PCore	概率缓冲 $P \llcorner [1,16]$ MXINT8-B16, 乒乓
ZBUF	PCore	GELU $\odot$ 的输出 $Z[51,512]$ MXINT8-B16
WFIFO	PCore	私有权重 FIFO, $512 \times 128\text{b}$
KVFIFO	PCore	KVB 相位匹配 FIFO, $128 \times 128\text{b}$
XFIFO	PCore	XBC 解耦 FIFO, $128 \times 256\text{b}$
DESC SRAM	CCU	一层一步的 MDESC 程序
EHU-ROM	GCore	time embedding LUT (20 KB) 与 RoPE LUT (25.5 KB)

2.5 缩写全表

2.5.1 模型与算法

缩写	全称	说明
VLA	Vision-Language-Action	视觉-语言-动作模型
VLM	Vision-Language Model	本设计中指 PaliGemma 2B
$\pi_o / \text{pi0}$	—	Physical Intelligence 的 VLA 基座模型
DiT	Diffusion Transformer	扩散式 Transformer
Flow Matching	—	连续时间生成建模，本设计的去噪范式
Euler step	—	一阶常微分方程积分, $x \leftarrow x + \Delta t \cdot v$
AR block	AutoRegressive block	掩码的分块单位，块内双向、块间因果
MQA	Multi-Query Attention	多 query head 共享单 KV head
GQA	Grouped-Query Attention	MQA 的一般化；本设计为 8:1

RoPE	Rotary Position Embedding	旋转位置编码
RMSNorm	Root Mean Square Normalization	均方根归一化，不减均值、无 bias
LayerNorm	Layer Normalization	层归一化（本模型不使用）
AdaRMS	Adaptive RMSNorm	自适应 RMSNorm（ $\pi_{0.5}$ 使用，本设计不支持）
GELU	Gaussian Error Linear Unit	本设计用 tanh 近似
SwiGLU / GeGLU	—	门控 FFN；本模型为 GeGLU 形式
KV cache	—	缓存的 key/value
prefill	—	首次前向，产生 KV cache
FlashAttention	—	分块 + 在线 softmax 的注意力算法
online softmax	—	单遍流式 softmax，维护 μ 与 ℓ

2.5.2 数值格式

缩写	全称	说明
MX	Microscaling	OCP 的ScaleScale浮点族
MXINT8	—	OCP 的 8 bit 整数型 MX 格式，k=32
MXINT8-B16	—	本设计的 profile，k=16，其余同 OCP
E8M0	8-bit Exponent, 0-bit Mantissa	纯 2 的幂Scale，偏置 127，255 保留为 NaN
Q1.6	—	1 符号位 + 1 整数位 + 6 小数位的定点解释
BF16	Brain Float 16	1+8+7，与 FP32 同指数范围
FP32	—	IEEE 754 binary32
INT8 / s8	—	8 bit 二补码；本设计用 $[-127, +127]$ 对称范围
RNE	Round to Nearest, ties to Even	架构唯一的舍入模式
ULP	Unit in the Last Place	末位单位
amax	—	块内最大绝对值
clamp / saturate	—	饱和截断

2.5.3 计算与体系结构

缩写	全称	说明
MAC	Multiply-ACcumulate	乘加
TOPS / GMAC / GFLOP	—	算力与工作量单位
GEMM	GEneral Matrix Multiply	通用矩阵乘
tile	—	16×16 的矩阵分块
stationary / streaming	—	驻留操作数 / 流动操作数
psum	partial sum	部分和
systolic array	—	脉动阵列
adder tree	—	加法树
Roofline	—	算力-带宽性能模型
AI, OI	Arithmetic / Operational Intensity	算术强度，FLOP/Byte
TP	Tensor Parallel	张量并行
WCET	Worst-Case Execution Time	最坏情况执行时间
PPA	Power, Performance, Area	功耗-性能-面积

2.5.4 电路与实现

缩写	全称	说明
RTL	Register Transfer Level	寄存器传输级
SVA	SystemVerilog Assertions	断言
FSM	Finite State Machine	有限状态机
LZC	Leading Zero Count	前导零计数
RMW	Read-Modify-Write	读改写
CDC	Clock Domain Crossing	跨时钟域
1RW / 1R1W	—	单读写口 / 一读一写口

RF	Register File	寄存器堆
skid buffer	—	打拍缓冲，用于切 ready 组合路径
AXI4 / APB	—	总线协议
pch	pseudo-channel	HBM 伪通道
URAM / BRAM / DSP48E2	—	FPGA 资源类型

2.5.5 本设计定义的模块与信号

缩写	说明
MDESC	Matrix DESCriptor，256 bit 静态描述符
pipe_en / pipe_en_q	全局计算使能（组合 / 寄存后）
all_ready / pipe_ready	全局就绪树
epoch	； pipe_en=1 的一个上升沿
SEG / SEGLOAD	权重段与段加载
SYNC_ID	计分板位号
POST_OP	描述符指定的后处理链
EXP_FOLD	附加的 2 的幂Scale折叠（如 QK 的 −4）
deq_tag_t	DEQACC 的 13 bit 目标标签
A_FIN	Attention 输出的归一化与量化算子
OACC_SCALE	FlashAttention 在VPU中对O进行的 α 重缩放操作

3 工作负载分析

3.1 模型结构

π_0 的动作专家是一个 18 层的 pre-norm Transformer，与 PaliGemma 共享注意力的 KV，但拥有独立的 Q 投影与 FFN。

位置	结构	备注
每层入口	RMSNorm，pre-norm，增益写作 $(1 + \gamma)$	γ 离线折进后继权重（§ 8.11）

注意力	MQA: 8 个 query head, 1 个共享 KV head, d_h = 256	KV 由 GCore 单份持有并广播
位置编码	RoPE, base = 10000,	prefix 的 K 在 VLM 侧已旋转
注意力后	残差加	FP32 归约, BF16 落盘
FFN 入口	RMSNorm	同上
FFN	GeGLU: $Z = \text{GELU_tanh}(X \cdot W_G) \odot (X \cdot W_U)$, 再 $Z \cdot W_D$	F = 4,096
FFN 后	残差加	
末层之后	final RMSNorm	γ 折进 action_out_proj

3.1.1 逐算子形状（单层）

算子	输入	权重	输出	说明
rms_a	[51,1024]	$\gamma_a[1024]$	[51,1024]	GCore 执行
q_proj	[51,1024]	[1024,2048]	[51,2048]	8 head $\times$ 256, 逐核 1 head
kv_proj	[51,1024]	$W_K[1024,256]$, $W_V[1024,256]$	各 [51,256]	8 核各持 32 列, gather 到 Gcore KV-Memory
rope_q, rope_k	—	LUT	—	保范变换
qk	$Q[51,256]$	$K^T[256,880]$	$S[51,880]$	逐核一个 head
softmax	S	—	$P[51,880]$	在线, 分 55 块
pv	$P[51,880]$	$V[880,256]$	$C[51,256]$	
o_proj	[51,2048]	[2048,1024]	[51,1024]	K 切分, CNET 归约
rms_f	[51,1024]	$\gamma_f[1024]$	[51,1024]	与残差加融合
gate_proj	[51,1024]	[1024,4096]	[51,4096]	N 切分, 每核 512 列
up_proj	[51,1024]	[1024,4096]	[51,4096]	同上, 与 gate 交织存放
gelu_mul	两个 [51,4096]	—	[51,4096]	逐元素
down_proj	[51,4096]	[4096,1024]	[51,1024]	K 切分, CNET 归约

3.1.2 步边界算子

算子	频次	形状
state_proj	每动作块一次	$[1,32] \times [32,1024]$
action_in_proj	每去噪步	$[50,32] \times [32,1024]$
time_embedding	每去噪步	LUT 查表 $[1,2048]$
action_time_mlp_in	每去噪步	$[50,2048] \times [2048,1024]$
action_time_mlp_out	每去噪步	$[50,1024] \times [1024,1024]$
final_rmsnorm	每去噪步	$[50,1024]$
action_out_proj	每去噪步	$[50,1024] \times [1024,32]$
Euler 更新	每去噪步	$[50,32]$

3.2 参数量与算力

3.2.1 参数量

单层权重（不含 RMSNorm 的 γ ， γ 见 § 8.11 的折叠处理）：

矩阵	形状	参数量
W_Q	1024×2048	2,097,152
W_K	1024×256	262,144
W_V	1024×256	262,144
W_O	2048×1024	2,097,152
W_gate	1024×4096	4,194,304
W_up	1024×4096	4,194,304
W_down	4096×1024	4,194,304
单层合计		17,301,504 (17.30 M)

层次	参数量
----	-----

18 层 Transformer	311,427,072 (311.4 M)
步边界投影 (state / action_in / time_mlp / action_out)	3,244,032 (3.24 M)
总计	314,671,104 (314.7 M)
每核每层	2,162,688

3.2.2 MAC 计算量统计

一个使能的 MXU 周期在每个 PCore 完成 $16 \times 16 = 256$ 次 INT8 乘法；8 核合计 2,048 MAC/使能周期。对矩阵 $[M,K] \times [K,N]$ ，N 均分到 8 核，理想阵列周期为 $M \times (K/16) \times (N/(8 \times 16))$ 。所有基线维度都是 16 的整数倍，因此无 K 尾巴、无 N 尾巴。

算子	代入	理想周期	MAC 校验
Q	$51 \times (1024/16) \times (2048/128)$	52,224	$52,224 \times 2,048 = 106,954,752$
K+V	$2 \times 51 \times (1024/16) \times (256/128)$	13,056	$13,056 \times 2,048 = 26,738,688$
QK	8 head $\times$ 51 $\times$ 55 块 $\times$ 16 key $\times$ (256/16) / 8 核	44,880	$44,880 \times 2,048 = 91,914,240$
PV	同 QK	44,880	$44,880 \times 2,048 = 91,914,240$
O	$51 \times (2048/16) \times (1024/128)$	52,224	$52,224 \times 2,048 = 106,954,752$
Gate+Up	$2 \times 51 \times (1024/16) \times (4096/128)$	208,896	$208,896 \times 2,048 = 427,819,008$
Down	$51 \times (4096/16) \times (1024/128)$	104,448	$104,448 \times 2,048 = 213,909,504$
层合计		520,608	1,066,205,184

物理注意力宽度 880 而逻辑宽度 867，padding 开销为 8 head $\times$ 2 矩阵 $\times$ 51 query $\times$ 13 padded key $\times$ 256 feature = 2,715,648 MAC。扣除后得逻辑 MAC **1,063,489,536 / 层 (1,063.5 MMAC)**。

层次	值
每层执行 MAC	1,066.2 M (含 padding)
每层逻辑 MAC	1,063.5 M
每去噪步	19.19 GMAC (18 层 + 步边界)
每动作块	193.03 GMAC = 386.07 GOP

3.3 KV cache 结构

项	值	说明
KV head 数	1	MQA，8 个 query head 共享
单层 prefix K	$816 \times 256 \times 1 \text{ B} + \text{ScaleScale}$	$816 \times 272 / 16 \approx 216.8 \text{ KB}$
单层 prefix V	同上	216.8 KB
单层 prefix KV		433.5 KB
18 层 prefix KV		7.80 MB（片外单份）
单层 suffix KV	$51 \text{（补至 64）} \times 256 \times 2 + \text{Scale}$	$\approx 34.8 \text{ KB}$ ，每步重算
动作块内的读取次数	10 步 $\times$ 18 层	每层每步读一次 prefix KV
动作块 KV 流量	$7.80 \text{ MB} \times 10 + \text{suffix 写回}$	$\approx 85 \text{ MB}$

prefix KV 在整个动作块期间**只读且不变**，这是双窗预取（§ 10.6）能够完全隐藏 KV 取数的前提。

3.4 Roofline：算术强度恒等于 M

这是本设计最重要的一条工作负载性质。

对任意权重矩阵 $[K, N]$ ，一次 GEMM 的：

- 计算量 = $2 \cdot M \cdot K \cdot N$ FLOP
- 权重字节数 = $K \cdot N \cdot (17/16) \text{ B}$ （MXINT8-B16 含Scale）

因此

$$AI = 2 \cdot M \cdot K \cdot N / (K \cdot N \cdot 17/16) = 32M/17 \approx 1.88 \text{ M [FLOP/B]}$$

算术强度只与 M 有关，与矩阵形状、与阵列大小、与层数全都无关。 M = 51 时 $AI \approx 96 \text{ FLOP/B}$ 。

3.5 注意力掩码结构

π_0 使用基于自回归块（AR block）的掩码。openpi 的 make_attn_mask 依据 ar_mask 的累积和分块：

```
cumsum = jnp.cumsum(mask_ar, axis=1)
attn_mask = cumsum[:, None, :] <= cumsum[:, :, None]
```

即 token i 能看到 token j 当且仅当 j 的块序号不大于 i 的块序号**同一块内为全连接（双向），而不是三角。**

AR 块	内容	ar_mask	cumsum	可见范围
0	prefix (3×256 图像 token + 48 文本 token)	全 False	0	块 0 (prefix 内部双向)
1	state token (suffix m = 0)	True	1	块 0, 1
2	action token (suffix m = 1…50)	首个 True, 其余 False	2	块 0, 1, 2

由此：

1. **suffix 不是因果的。** action token 之间是双向全连。
2. **state token 看不到 action token。**
3. **prefix 内部对 suffix 而言恒可见。** 基线 PREFIX_VALID 全 1。

3.5.1 物理块上的掩码退化

B_kv = 16 时 suffix 占物理块 51–54，共 4 个块。令

jj = ((blk - 51) << 4) + j # suffix 内的全局列号，0..63

masked = (jj >= 51) OR (m == 0 AND jj >= 1)

前一项是 padding 列，后一项是 state token 不看 action token。MASK_EN 仅在 blk >= 51 时置位。

块	jj 范围	该块的掩码退化形式
51	0–15	masked = (m==0 AND jj>=1)，即 state 只看 jj = 0（自身）
52	16–31	masked = (m==0)，整行屏蔽 state
53	32–47	masked = (m==0)
54	48–63	masked = (jj>=51) OR (m==0)，含 13 列 padding
0–50	—	基线全可见；PREFIX_RUNTIME_EN=1 时按 PREFIX_VALID[jj]

掩码必须在 rowmax 之前施加。 padding 列与被屏蔽列一律置 $-\infty$ ，否则会污染 μ 与 ℓ 。实现上用 `score = valid ? score : NEG_INF_FP32`。

3.7 张量清单与生命周期

--	--	--	--

张量	形状	存储格式	生命周期 / 归属
RESID	51×1024	BF16	GCore；跨算子与跨层持久
XHAT + E_a	$51 \times 1024, 51 \times 64$	INT8 + E8M0	GCore；从 RMS Pass B 到消费它的投影组
Q_h	51×256 / 核	INT8 + E8M0	PCore Q-Memory；一层的 Attention 期间有效
K/V prefix	$\leq 816 \times 256$ / 层	INT8 + E8M0	GCore KV SRAM；动作块内只读
K/V suffix	51×256 / 层	INT8 + E8M0	GCore KV suffix ；每去噪步重算
S / P	51×16 / 块	FP32 / BF16 / INT8+E8M0	PCore 乒乓暂存；至多两代
$\mu / \ell / \alpha$	各 51 标量	FP32	PCore Attention 状态；层 Attention 入口复位
OACC	51×256	FP32	PCore；跨全部 55 个块存活
Z + E_z	51×512 / 核	INT8 + E8M0	PCore；到 down_proj 消费为止
x_t / v_t	50×32	BF16 存 / FP32 算	GCore EHU；跨去噪步边界
time emb LUT	10×2048	BF16	GCore ROM，20 KB，只读
RoPE LUT	$51 \times 128 \times 2$	BF16	GCore/PCore ROM，25.5 KB，只读

4 数值架构

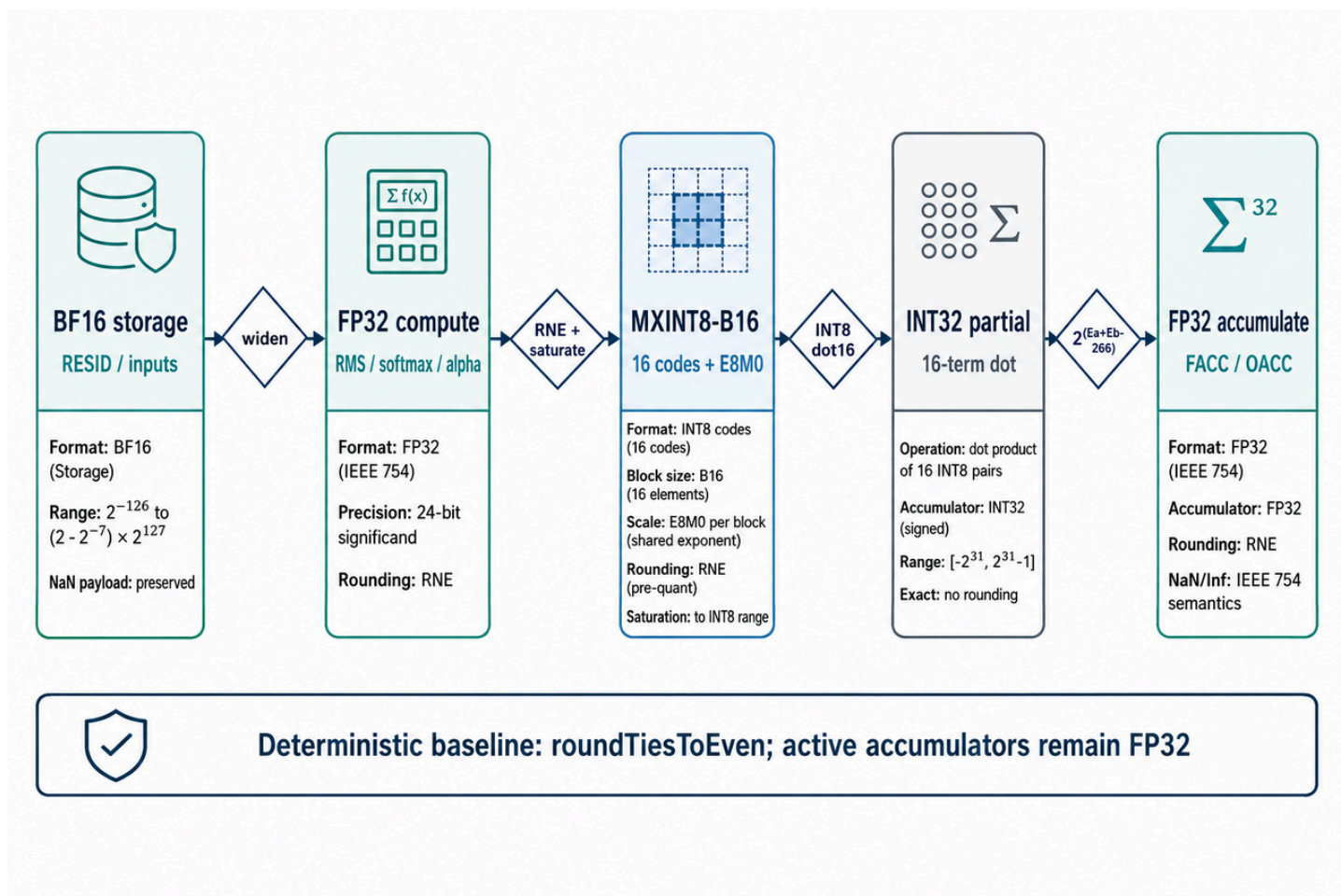


图 4-1 精度域与强制转换边界

4.1 精度分配策略

对象 / 运算	存储	计算	规则
残差流、state/action 输入、time embedding	BF16	被非 MX 内核消费时 FP32	FP32→BF16 用 roundTiesToEven
MX 权重与激活	MXINT8-B16	INT8 乘积，INT32 16 项部分和	每块部分和先反量化，再做 FP32 跨块累加
FACC、OACC、softmax 的 $\mu/\ell/\alpha$ 、RMS 平方和	FP32	FP32	活跃数值状态， 不得降级为 BF16
CNET 归约	FP32 流	FP32 加法树	最终残差落盘时舍入到 BF16
RoPE / GELU / exp / reciprocal / rsqrt	BF16	FP32 VPU	须满足 § 4.11 的逐函数 ULP/误差界
Euler 状态更新	BF16 输入/输出	FP32 乘加	只在输出存储边界舍入一次
α (在线 softmax 重缩放)	FP32	FP32 乘法 (VPU)	

4.2 MXINT8-B16 定义

本项目格式命名为 **MXINT8-B16**。它保留 OCP MXINT8 的值解释、E8M0 Scale编码、二补码元素编码、隐含的 2^{-6} 元素Scale、强制 roundTiesToEven 转换与饱和转换行为。**相对 OCP 具体格式的唯一偏离是Scale块大小 $k = 16$ 而非 $k = 32$ 。**

字段	编码	值解释
块ScaleScale E	8 bit E8M0; E = 255 保留为 NaN	$X = 2^{(E-127)}$, $E \in [0, 254]$
元素码数据 q	有符号 INT8 二补码	$P = q \times 2^{-6}$; 默认对称范围 $q \in [-127, +127]$
解码元素	16 个码共享一个 E	$v \boxtimes q \times 2^{(E-133)}$
全零块	E = 0, 所有 q = 0	规范零 (E8M0 本身没有零Scale)
有限溢出	保号饱和到 $q = \pm 127$	
NaN 输入	CSR 可选: 故障 或 规范 NaN 块	

有效位宽 = $8 + 8/16 = 8.5 \text{ bit / 元素}$ 。

单位	组成	字节数
一个 MXINT8-B16 块	16 个元素	16 B 数据 + 1 B Scale = 17 B
一个 MXU tile	16×16 元素 = 16 个块	256 B 数据 + 16 B Scale = 272 B

4.4 量化（编码器）

4.4.1 指数的计算：floor 型

给定块内最大绝对值 $a_{\max}$ (FP32) , 取

代码块

```
1  e = floor( log2(a_max) )
2  E = clamp( e + 127, 0, 254 )
```

设 $a_{\max} = m_x \cdot 2^p$, $m_x \in [1, 2)$, 则 $\text{floor}(\log_2(a_{\max})) = p$, 而 FP32 的有偏指数字段恰为 $e_x = p + 127$ 。因此

$E = \text{clamp}(e_x, 0, 254)$

实现即直接取 FP32 的指数字段，再做一次边界 clamp

4.4.2 量化

代码块

```
1 q[i] = clamp( RNE( v[i] * 2^(133 - E) ), -127, +127 )
```

被量化的数以 BF16 或 FP32 存放，乘 $2^{(133-E)}$ 是**指数字段加法**，随后做定点舍入到整数Scale饱和器（clamp 到 ± 127 ）在此是必需的正常路径逻辑。

4.4.3 编码器伪码

代码块

```
1 function encode_mx_b16(v[16]):
2     if any_is_nan(v): return exception_policy(NAN)
3     if all(v == 0):
4         return {E = 0, q[0:15] = 0}
5     amax = max_i(abs(v[i]))
6     e_x = fp32_exponent_field(amax) # 含偏置 127
7     E = clamp(e_x, 0, 254)         for i in 0..15:
8     scaled = v[i] * 2**((133 - E))
9     q[i] = sat_sym_127( round_ties_to_even(scaled) ) # 基线下会在正常数据上触发
10    return {E, q}
```

4.5 反量化与点积

4.5.1 推导

对码 q_a 配Scale E_a ， $\hat{a} = q_a \times 2^{(E_a-133)}$ ；同理 $b \boxtimes q_b \times 2^{(E_b-133)}$ 。16 项点积：

代码块

```
1 sum_i( a_hat_i * b_hat_i )
2 = sum_i( (q_a_i * 2^(E_a-133)) * (q_b_i * 2^(E_b-133)) )
3 = sum_i( q_a_i * q_b_i ) * 2^(E_a + E_b - 266)
4 = INT32_PSUM * pow2( E_a + E_b - 266 )
```

真实指数偏移是 -266 ，不是 -254 。差的 -12 是两个隐含 Q1.6 元素Scale的乘积。

4.6 $1/\sqrt{d_h}$ 的位精确折叠

注意力的缩放因子恰好是 2 的幂：

$1/\text{sqrt}(256) = 1/16 = 2^{-4}$

因此可以在量化 Q 时直接令 $E_q \leftarrow E_q - 4\text{Scale}$ 。

4.7 确定性与异常值

- 架构舍入模式为 **roundTiesToEven**
- E = 255 只在所选 NaN 策略允许时传播规范 NaN 块；有限内核永不生成 E = 255。
- MXINT8 不产生有符号负零。BF16 的有符号零遵循 IEEE 行为。
- FP32 累加顺序由 tile 顺序与归约树拓扑固定**

4.8 各张量的块方向

块方向必须是该张量参与的 GEMM 的**归约维**——这样一个块才对应加法树的一次完整归约。

张量	参与的 GEMM	归约维	块方向	备注
$X \hat{\otimes} X^{\text{HAT}}$	q/kv/gate/up_proj	$K = d_{\text{model}}$	沿输入通道	每行 64 个块
$W_Q / W_K / W_V$	同上	$K = d_{\text{model}}$	沿输入通道	离线打包
$Q \hat{\otimes}$	QK	$K = d_h$	沿输入通道	每行 16 个块
$K \hat{\otimes}$	QK	$K = d_h$	沿输入通道	prefix 在 VLM 侧已按此打包
$P \hat{\otimes}$	PV	$K = \text{key}$	沿 seq (key)	一个块恰为一个 KV 块，见 § 7.7
$V \hat{\otimes}$	PV	$K = \text{key}$	沿 seq	故 V 的量化需要 16 token 的暂存（§ 8.2.5）
$\hat{O}$	o_proj	$K = d_h$ （拼接后 2048）	沿输入通道	A_FIN 产出
W_O	o_proj	$K = 2048$	沿输入通道	K 切分，每核 256 行
$\hat{z}$	down_proj	$K = d_{\text{ff}}$	沿输入通道	每核 512 列 → 32 个块
$W_{\text{gate}} / W_{\text{up}}$	gate/up_proj	$K = d_{\text{model}}$	沿输入通道	须交织存放，见 § 10.6.1
W_{down}	down_proj	$K = d_{\text{ff}}$	沿输入通道	K 切分

4.9 RMSNorm

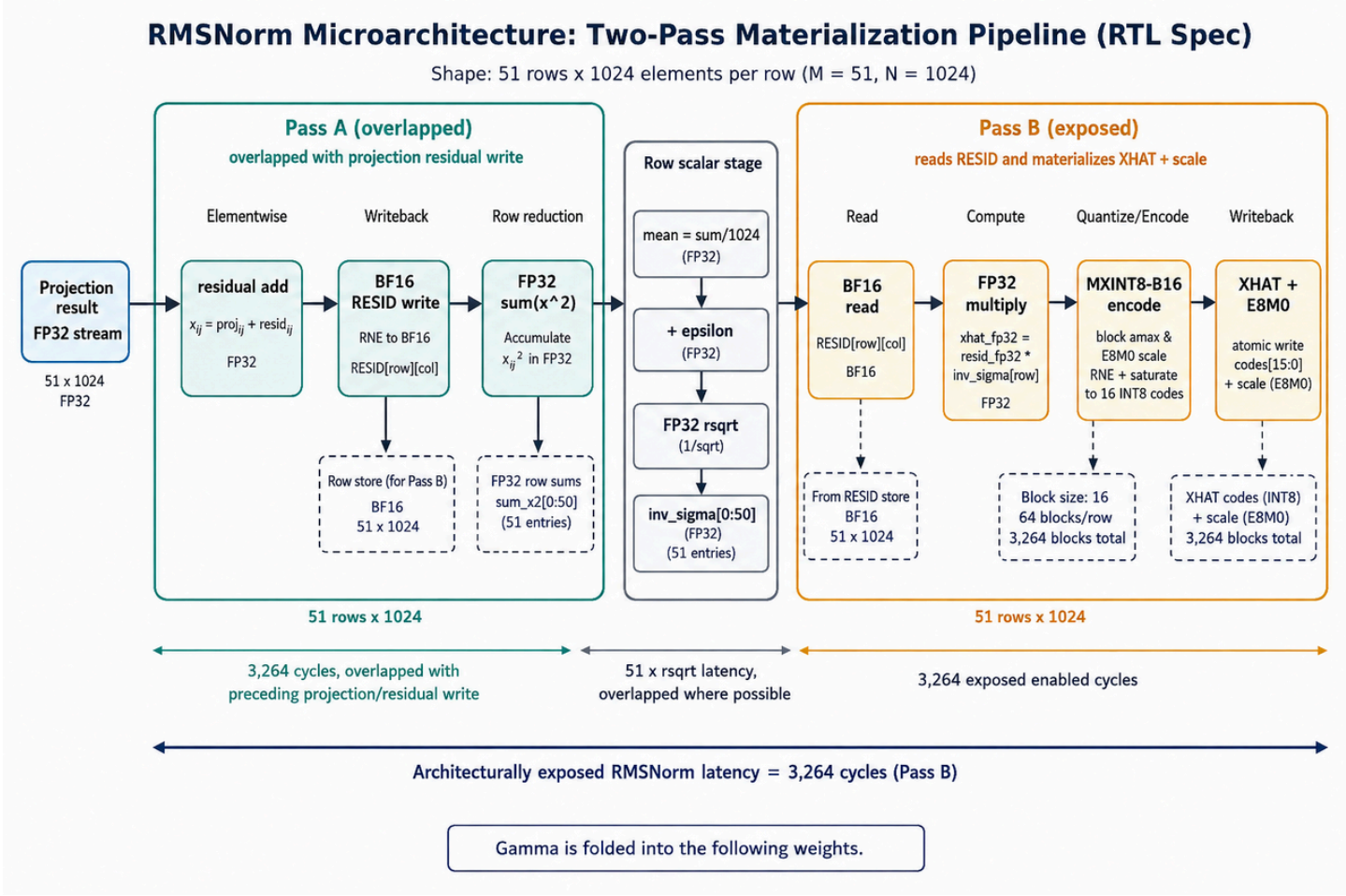


图 4-3 两遍物化保留真实的归一化 MXINT8-B16 值

对 $x \in \mathbb{R}^{1024}$ ， $\text{inv_sigma} = \text{rsqrt}(\text{mean}(x^2) + \text{eps})$ 在 FP32 中计算。 γ 离线折进后继投影权重 (§ 8.11)。

- Pass A 与前一算子的 CNET 残差写融合，累积 FP32 平方和。
- Pass B 读 BF16 残差，在 FP32 中乘以 inv_sigma，把真实的归一化值转换为 MXINT8-B16。

4.10 逐算子精度分配

算子	输入格式	计算	输出格式
RMS 平方和	BF16 → FP32	FP32 加法	FP32 标量
rsqrt	FP32	FP32 + 一次细化	FP32
RMS Pass B	BF16 × FP32	FP32 乘	MXINT8-B16
q/kv/gate/up/o/down_proj	MXINT8-B16	INT8 → INT32 → FP32	FP32 (FACC)
RoPE		FP32 乘加	MXINT8-B16

	FP32 中间 × BF16 表		
QK	MXINT8-B16	同上， EXP_FOLD=-4	FP32 (SBUF)
掩码	FP32	选择 $-\infty$	FP32
rowmax / μ 更新	FP32	FP32 比较	FP32
exp	FP32	SFU	FP32 → MXINT8-B16 (PBUF)
PV	MXINT8-B16	INT8 → INT32 → FP32	FP32 (OACC RMW)
α 重缩放	FP32	VPU FP32 乘	FP32 (OACC)
A_FIN, O-update	FP32 ÷ FP32	reciprocal + 乘	MXINT8-B16
CNET 归约	FP32	FP32 平衡树	FP32 → BF16 存储
GELU $\odot$	FP32	SFU + FP32 乘	MXINT8-B16 (ZBUF)
残差加	FP32 + BF16→FP32	FP32 加	BF16
Euler 更新	BF16 → FP32	FP32 乘加	BF16

4.11 CNET 部分和的传输精度

CNET 的归约过程采用 FP32，结果写入 GCore 后以 BF16 存储。

顶层架构：1 个 GCore + 8 个 PCore

GCore 持有全局共享张量（残差流、XHAT、KV），PCore 持有私有权重与头本地中间量

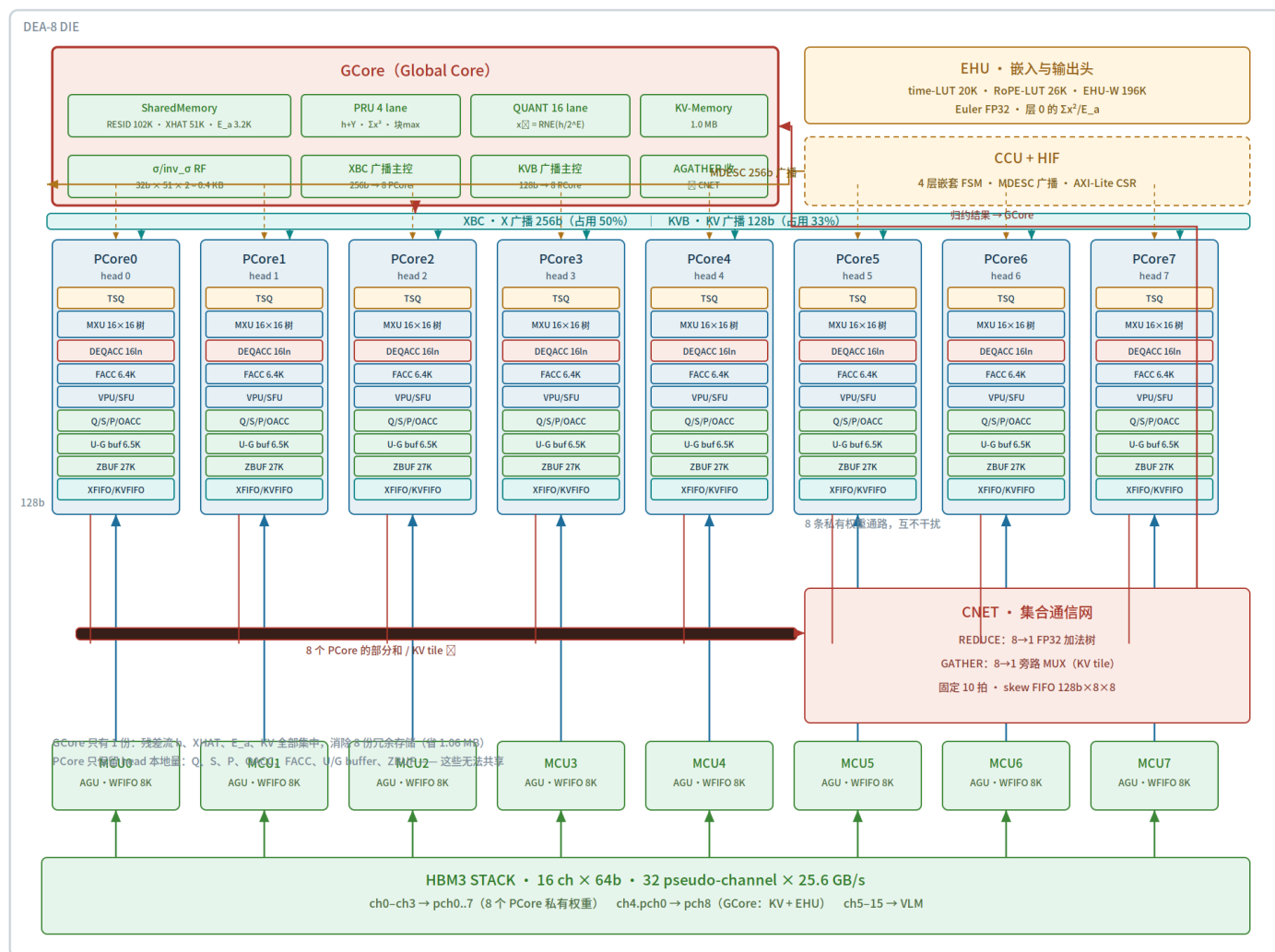


图 5-1 顶层架构：1 个 GCore + 8 个 PCore + 三条跨核通路

5. 顶层架构

5.1 GCore / PCore 的划分依据

划分依据是一个简单判据：

一个张量若在 8 个核上是同一份内容，就放进 GCore；若是每核不同的分片，就放进 PCore。一个运算若 8 个核会得出相同结果，就在 GCore 做一次；若结果各核不同，就在 PCore 各自做。

归属	张量 / 运算
GCore	残差流 $h[51,1024]$
GCore	$XHAT[51,1024] + E_a$

GCore	RMSNorm (Σx^2 、rsqrt、量化)
GCore	prefix/suffix KV
GCore	time embedding、Euler 更新、小投影 (EHU)
PCore	W_Q/W_K/W_V/W_O/W_gate/W_up/W_down 的分片
PCore	Q $\hat{\alpha}$ 、OACC_i、 $\mu/\ell/\alpha_i$ 、 $\hat{z}_i$ 、FACC_i
PCore	S、P

5.2 顶层模块清单

模块	数量	职责	主要存储 / 接口
GCore	1	共享残差、RMS/量化、prefix/suffix KV、归约、EHU	RESID、XHAT、E_a、KV 窗口、标量 RF
PCore	8	一个 query head 与一路私有权重流	MXU、DEQACC、VPU/SFU、Q/S/P/O/Z/FACC
MCU	8	私有 HBM 权重取址与 tile 组装	256 bit AXI 读 $\rightarrow$ 17 $\times$ 128 bit 内部条目
CCU	1	静态描述符发射、屏障、错误处理	DESC SRAM 与计分板
CNET	1	8 $\rightarrow$ 1 FP32 归约与 8 $\rightarrow$ 1 gather	与计算域同步全局停顿
XBC / KVB	各 1	广播归一化激活与共享 KV tile	ready/valid，同一全局计算
HIF	1	CSR / 命令队列 / 中断	APB 或 AXI-Lite
DBG	1	性能计数器与 trace 捕获	always-live 域

5.2.1 顶层实现层次

代码块

```
1  dea8_top
2  +-- dea8_ctrl_wrap      # 命令/CSR/中断/CCU
3  +-- dea8_pipe_ctrl     # 全局就绪树与 pipe_en
4  +-- dea8_hbm_wrap[0:8]  # 9 个 AXI 主与 CDC FIFO
5  +-- dea8_mcu[0:7]      # 私有权重补给
6  +-- dea8_pcore[0:7]    # query head 计算核
7  |   +-- tsq / local_sram / mxu / deqacc / vpu / sfu
8  +-- dea8_gcore         # RESID/XHAT/KV/RMS/quant/EHU
9  +-- dea8_xbc / dea8_kvb # 锁步广播
```

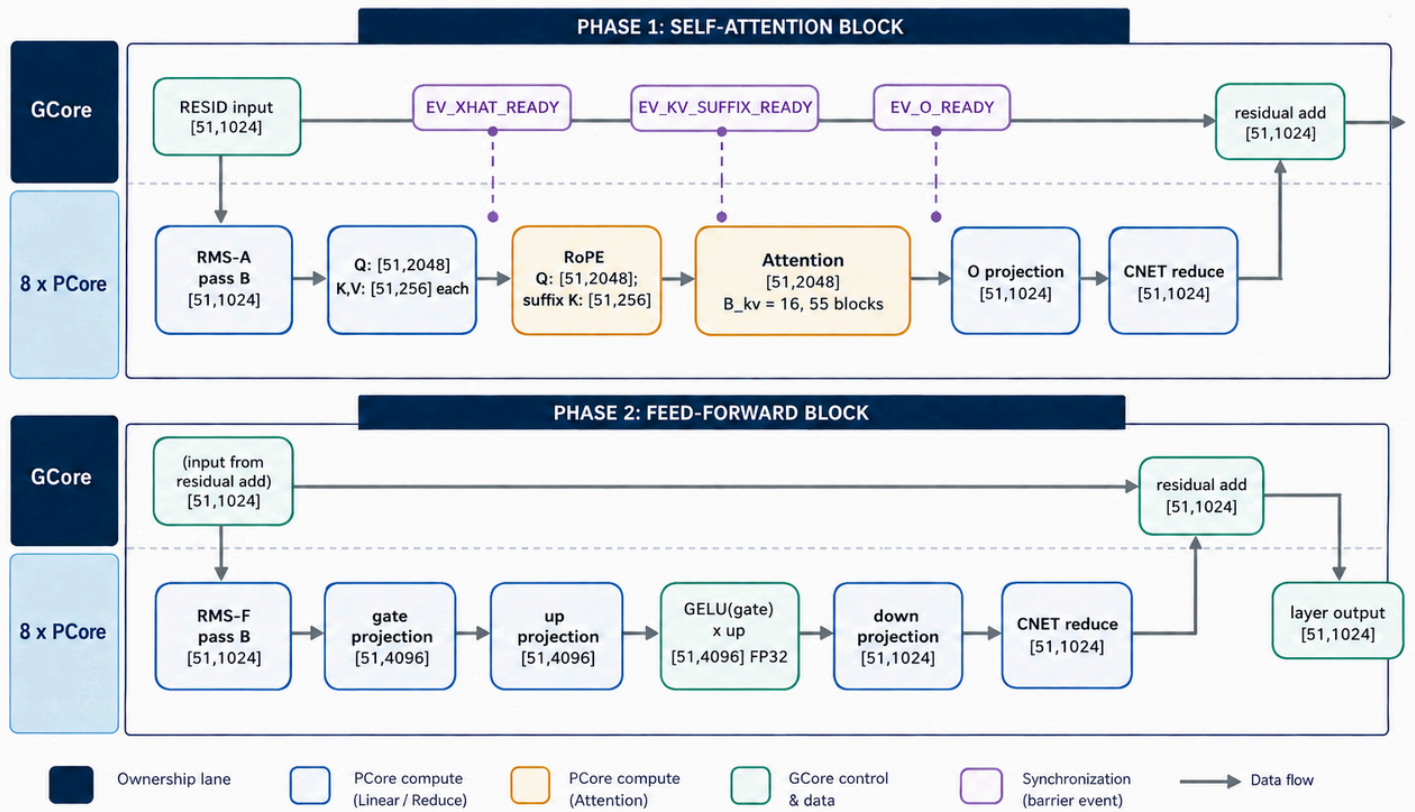
```
10    +-- dea8_cnet          # 确定性 reduce/gather
11    +-- dea8_dbg_telemetry # 计数器与 trace
```

5.3 张量并行映射

八个线性算子按不变量 I4 分为两组：**q/k/v/gate/up 按 N 维（输出列）切分，o/down 按 K 维（输入行）切分。**

算子	切分维	PCore i 持有的权重	PCore i 产出	通信
q_proj	N（按 head）	W_q[:, 256i : 256i+256]	Q_i[51,256]	无
k_proj	N	见 § 8.2.2 的两段列映射	K_i[51,32]	CNET GATHER
v_proj	N	W_v[:, 32i : 32i+32]	V_i[51,32]	CNET GATHER
$QK^T / P \cdot V$	head 本地	—	O_i[51,256]	KVB 广播
o_proj	K（按行）	W_o[256i : 256i+256, :]	部分和[51,1024]	CNET REDUCE
gate_proj	N	W_g[:, 512i : 512i+512]	G_i[51,512]	无
up_proj	N	W_u[:, 512i : 512i+512]	U_i[51,512]	无
down_proj	K（按行）	W_d[512i : 512i+512, :]	部分和[51,1024]	CNET REDUCE

One transformer layer flow with two residual branches



为什么这样配对 (I4)：列并行的输出天然的分片的，直接喂给下一个行并行算子作为它的输入分片，中间无需任何通信；行并行的输出才需要归约。因此每个子块（Attention 一次、MLP 一次）只需 **1 次集合通信**，且发生在 GEMM 之后，可以与后续 tile 的计算流水重叠。

5.5 三条跨核通路

通路	方向	位宽	载荷	占空	说明
XBC	1 → 8	256b + 16b Scale旁带	XHAT tile	50%	见 § 5.5.1
KVB	1 → 8	128b + 8b Scale旁带	K/V tile	33.3%	仅 Attention 段
CNET	8 → 1 → 8	512b (leaf 8×512b)	FP32 部分和 / KV tile	12.5%	REDUCE 与 GATHER 时分复用

5.5.1 XBC 带宽核算

读取 XHAT 的算子共四个。由于 8 个 PCore 执行同一份 MDESC，在同一拍处于同一个 (n, k)，因此需要的是同一个 XHAT tile——**一次广播服务 8 核，带宽是 16 B/拍而非 8 × 16 B/拍。**

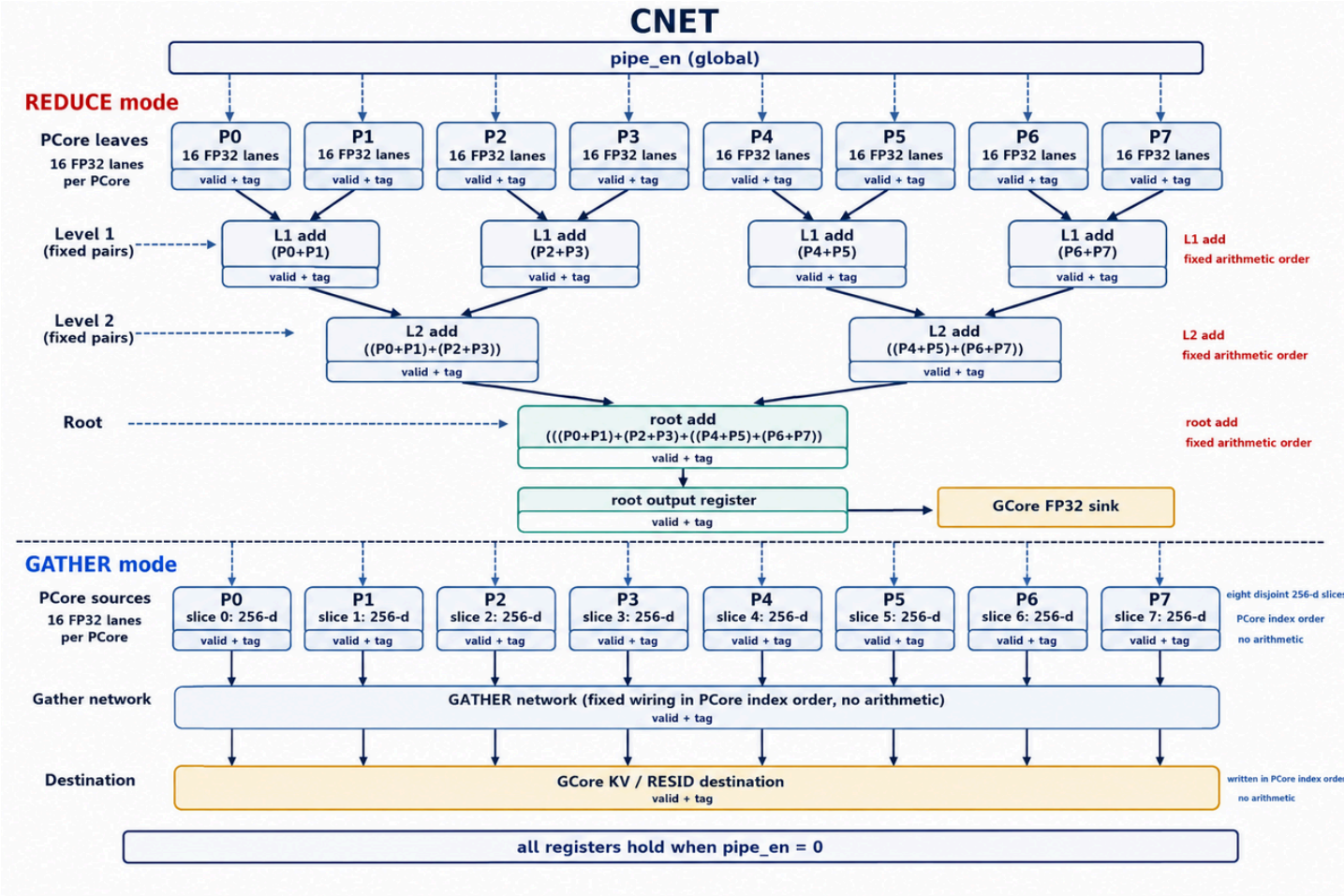
算子	Kt	Nt	拍数	是否读 XHAT
kv_proj	64	4	13,056	是
q_proj	64	16	52,224	是
gate_proj	64	32	104,448	是
up_proj	64	32	104,448	是
o_proj	16	64	52,224	否（读 OACC）
down_proj	32	64	104,448	否（读 ZBUF）
合计读 XHAT			274,176（层的 52.6%）	16 B/拍

若 XBC 为 128b，则在这四个算子期间是 100% 占用、零余量；取 256b 时占用 50%，有真实裕度。本设计取 256b。

XFIFO：把耦合降级为流控问题

XBC 只能在所有 8 个 PCore 都有接收空间时推进，因此某个核长时间停摆会拖住全部。缓解手段是每核一个 XFIFO（256b × 128 = 4 KB，提供 128 拍解耦）。

5.6 互连事务规则



通路	负载	流控规则	定序
XBC	每 256 bit beat 携带两个 16 元素激活块加Scale旁带	仅当每个目的端都能接收该周期时才发起	所有 PCore 在同一逻辑块上
KVB	每内部条目 16 个 INT8 加一个 E8M0	广播到 8 个 KVFIFO；FIFO 是突发性，不是相位纠正	块、特征、先 K 后 V 的顺序由描述符固定
CNET REDUCE	8×16 FP32 lane $\rightarrow$ 16 FP32 lane	平衡固定树，全局使能	PCore 序号顺序固定以保证可复现性
CNET GATHER	8 个不相交的 256 维分片	按地址序写出一个逻辑 KV 向量	无算术；字节 lane 映射须断言

5.7 存储器

全部为同步读（地址寄存、数据下一拍出）。只使用三种基元：sram_1rw、rf_1r1w、fifo_sync。

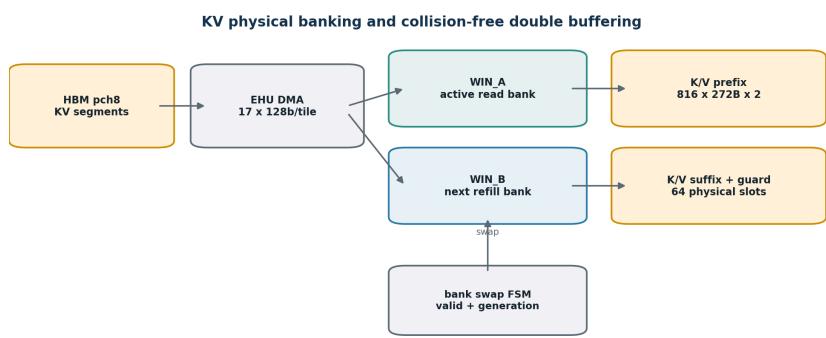


图 10-1 KV 双缓冲分体、补给与活动代切换（源：V1.2 Fig.10-1）

5.7.1 GCore SRAM

存储器	格式 / 组织	容量	端口	用途
RESID	BF16, 256b $\times$ 3,264	102 KiB	1RW + 字节使能	h[51,1024]
XHAT	INT8 , 128b $\times$ 3,264	51 KiB	1RW	归一化并量化后的 $X\boxtimes$
E_a	E8M0, 8b $\times$ 3,264	3.19 KiB	1RW	[51,64] ScaleScale
sq / inv_σ RF	FP32, 32b $\times$ 51 $\times$ 2	0.40 KiB	1R1W	Σx^2 与 rsqrt 结果
KV WIN_A / WIN_B	两个独立的 prefix 窗口	2 $\times$ 433.5 KiB	独立 1RW 分体	层 L 读 / 层 L+1 预取 [816, 256] x2
KV suffix / guard	51 个逻辑 token 加对齐	$\approx$ 55 KiB	专用分体或不冲突分区	每步重算 [51, 256]

EHU LUT ROM	time emb 40 KB + RoPE 25.5 KB	65.5 KiB	ROM	§ 8.10.2、§ 8.12
GCore 合计	含控制余量	≈ 1.15 MiB		宏选型受 SRAM 编译器约束

5.7.2 PCore SRAM（每核）

存储器	格式	Width × Depth	容量	端口	用途
QOZ-BUF	INT8	128b × 1632	25.50 KiB	1RW	分时复用为Q, O, Z存储
E_qoz	E8M0	8b × 1632	1.59 KiB	1RW	同上复用为 E_o
SUG-BUF	FP32	512b × 51 × 2	6.38 KiB	各 1RW	S[51,16]乒乓, U/G-Tile
PBUF	INT8	128b × 51 × 2	1.59 KiB	1R1W	P[51,16], 乒乓且同时读写
E_p	E8M0	8b × 51 × 2	0.10 KiB	1R1W	随 PBUF 乒乓
OACC	FP32	512b × 816	51.00 KiB	1R1W	Attention 累加器
FACC	FP32	512b × 51 × 2	6.38 KiB	1R1W	全部 GEMM 共用, 乒乓
WFIFO	INT8+E8M0	128b × 512	8.00 KiB	FIFO	1,536 拍供给
KVFIFO	INT8+E8M0	128b × 128	2.00 KiB	FIFO	相位匹配, 见 § 10.5
XFIFO	INT8	256b × 128	4.00 KiB	FIFO	128 拍解耦
μ/ℓ/α RF	FP32	32b × 51 × 4	0.80 KiB	1R1W	μ、ℓ 各一份, α 两份
U/G tile buf	FP32	512b × 51 × 2	6.38 KiB	1RW	GELU⊙ 的两个输入
ZBUF	INT8	128b × 1,632	25.50 KiB	1RW 分体	GELU⊙ 输出
E_z	E8M0	8b × 1,632	1.59 KiB	1RW	—
标量状态与余量	—	—	≈ 0.70 KiB	RF	标签、ECC、遥测
PCore 逻辑合计			≈		
实现分配			160 KiB		约 32 KiB 宏/走线余量

6 核内微架构

图 6-1 PCore 内部模块、存储与流端点（源：V1.2 Fig.6-1）

6.1 PCore 数据通路

PCore 的数据通路由 TSQ 统一定序，包含五个流水段：取数、矩阵乘、反量化累加、向量后处理、写回。

段	功能	延时 / 吞吐	停顿行为
TSQ / 地址	展开描述符并生成本地/广播地址	1 拍	保持计数器与标签
SRAM 读	读激活与 stationary 数据	1 拍	保持地址/输出寄存器
MXU	16×16 INT8 乘积与 16 项归约	6 拍延时；256 MAC/拍	保持每一个流水寄存器
DEQACC	施加 E_a+E_b−266 并 FP32 累加	5 拍延时；16 lane/拍	保持标签、FACC/OACC 写
VPU	BF16/FP32 向量运算与量化	16 lane/拍	保持向量状态与写使能
SFU	exp、reciprocal、rsqrt 近似	标称 2 lane/拍	保持请求/响应流水

6.1.1 位宽约定

全部位宽由阵列边长推导，改变 TILE_R / TILE_C 时同步缩放：

通路	位宽	组成	推导
stationary（权重 / K / V）	128 bit	16 × INT8	TILE_R × 8
stationary Scale	128 bit	16 × E8M0	TILE_C × 8
activation	128 bit	16 × INT8	TILE_R × 8
psum	512 bit	16 × INT32	TILE_C × 32
VPU 通路	512 bit	16 lane × FP32	TILE_R × 32
残差通路	256 bit	16 × BF16	TILE_R × 16
CNET	512 bit	16 × FP32	TILE_R × 32（D9）
KVB	128 bit	16 × INT8	TILE_R × 8
XBC	256 bit	32 × INT8	见 § 5.5.1

6.2 MXU：16×16 乘加树

6.2.1 结构

项	规格	说明
组数 × 每组乘法器	$16 \times 16 = 256$	第 n 组计算输出通道 n
激活广播	128b, $1 \rightarrow 16$	物理尺度约 0.02 mm ² ，插一级 buffer
加法树	4 级，INT32	16 输入 $\rightarrow$ 1 输出，平衡树， 结合次序固定
MXU_LAT	6 拍	1 乘 + 4 加 + 1 输出寄存
吞吐	51 拍 / tile	与 K、N 无关（M = 51）
权重装载	16 拍	$256 \text{ B} \div 16 \text{ B/拍}$ ；窗口是前一 tile 的 51 拍
权重切换	全局 1 拍	双缓冲寄存器整体 swap
输出	$512\text{b/拍} = 16 \times \text{INT32}$	直送 DEQACC

6.3 逐拍流水线

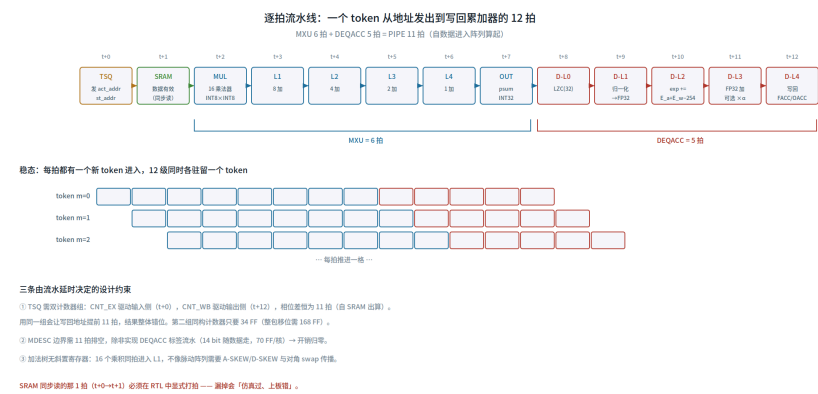


图 6-3 一个 token 从地址发出到写回累加器的 12 拍（源：V1.0 图 5）

拍	单元	动作	说明
t+0	TSQ	发出 act_addr / st_addr	纯组合，由 CNT_EX 驱动
t+1	SRAM	Q-Memory / PBUF / XFIFO 数据有效	同步读的 1 拍，RTL 必须显式打拍
t+2	MXU · MUL	16 个乘法器： $\text{INT8} \times \text{INT8} \rightarrow \text{INT16}$	16 组并行

t+3	MXU · L1	加法树第 1 级（8 个加法）	INT16 → INT20
t+4	MXU · L2	第 2 级（4 个加法）	—
t+5	MXU · L3	第 3 级（2 个加法）	—
t+6	MXU · L4	第 4 级（1 个加法）	—
t+7	MXU · OUT	psum 寄存输出，512b = 16 × INT32	§ 4.5.2 的范围核验
t+8	DEQACC · L0	LZC(32) 前导零计数	16 lane 并行
t+9	DEQACC · L1	barrel shift → FP32 尾数与初始指数	—
t+10	DEQACC · L2	exp += E_stream + E_stat[j] − 266 + EXP_FOLD	每 lane 一个 8 位加法器
t+11	DEQACC · L3	FP32 加	与累加器读值相加（ 不含 α 乘，裁决 D4 ）
t+12	DEQACC · L4	写回 FACC 或 OACC	1R1W，同拍读旧值写新值

PIPE_DRAIN = MXU_LAT + DEQACC_LAT = 6 + 5 = 11 拍（自数据进入阵列算起）。

6.3.1 由流水延时导出的三个约束

编号	约束	说明
①	TSQ 需 双计数器组	CNT_EX 驱动输入侧（t+0），CNT_WB 驱动输出侧（t+12），相位差恒为 11 拍。用同一组会让写回地址提前 11 拍、结果整体错位。第二组同构计数器只需 34 个触发器（整包移位需 168 个）
②	MDESC 边界需 11 拍排空	除非实现 DEQACC 标签流水（§ 6.4.3），届时开销归零
③	全局 pipe_en 而非逐级握手	加法树没有弹性，任何一拍的空泡都会破坏 tile 内 16 项的对齐，只能整体冻结。副作用是失速分析变得平凡

6.4 DEQACC 与 FACC

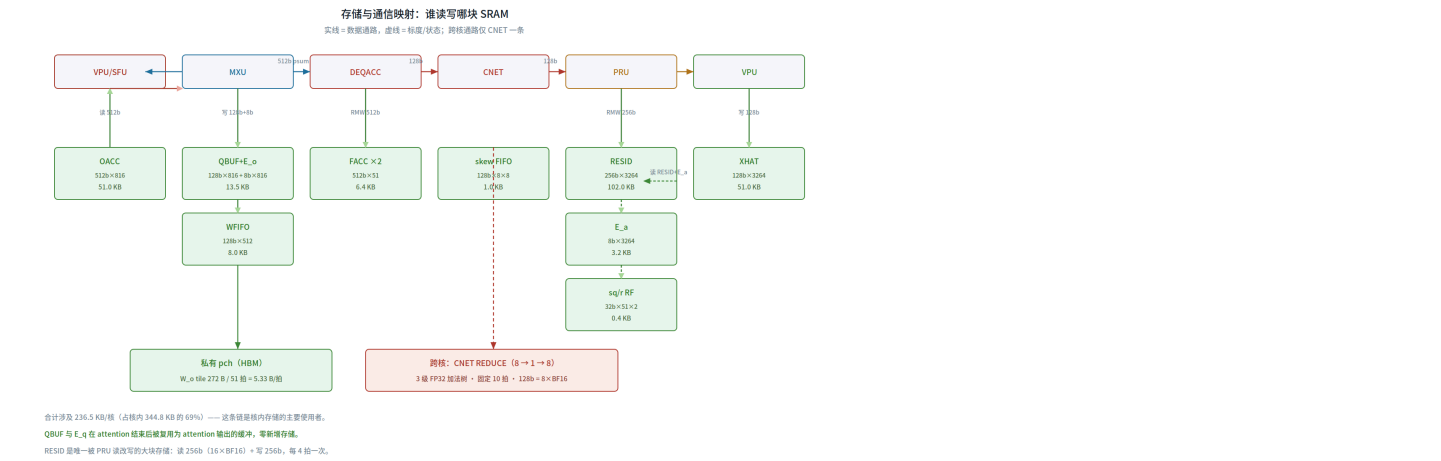


图 6-4 DEQACC 在数据通路中的位置与存储映射（源：V1.0 图 6）

6.4.1 为什么需要一个专用单元

由不变量 I5，MXINT8 的 Scale 沿归约维每 16 个元素变一次，跨 tile 的 INT32 累加在数学上不成立，必须逐 tile 反量化。反量化速率因此从”每个 N-tile 一次”提高到”每拍 16 个元素”，**提高了 64 倍**。

若 Scale 是一般浮点数，这需要 16 个 FP32 乘法器。而 E8M0 使反量化退化为指数加法，因此**一个 5 级、无乘法器的专用单元即可**。

裁决 D4 的连带收益 V1.0 曾在 DEQACC 的 L3 放一个条件乘用于 Attention 的 α 重缩放。V2.0 把 α 移交 VPU 后，DEQACC 回归为**纯加法器 + 移位器**结构，恰好与本节自身的论证一致。

6.4.2 lane 结构与资源

级	功能	实现	说明
L0	前导零计数	LZC(32)	INT32 归一化的第一步
L1	归一化	barrel shifter	得到 FP32 尾数与初始指数
L2	指数调整	8 位加法	$s[j] = E_stream + E_stat[j] - 266 + EXP_FOLD$
L3	累加	FP32 加法器	无乘法器（D4）
L4	写回	FACC / OACC 1R1W	同拍读旧值写新值

每 lane 约 2,800 门，16 lane 约 45 k 门/核，8 核约 360 k 门，估计 0.15 mm² @ 7 nm。

指数路径的一个性质值得注意：**同一拍内 16 个 lane 共享同一个 E_stream**（同一个 token、同一个 K-tile），而各自使用不同的 E_stat[j]（16 个输出通道）。因此每 lane 只需要一个 8 位加法器，E_stat 随 stationary tile 从 WFIFO 或 KVB 一同到达，无需独立访存流。

6.4.3 标签流水：把 MDESC 切换开销降为 0

保守实现下，两条 MDESC 之间需要 11 拍排空后才能重配置。但结果流是逐拍严格有序落地的：MDESC i 的最后一个结果在 t+11，MDESC i+1 的第一个在 t+12，两者相邻但不重叠。因此不需要排空，只需让目标随数据走：

```
typedef struct packed {
    logic [1:0] acc_sel; // FACC_A / FACC_B / OACC / SBUF
    logic [9:0] acc_addr; // 累加器地址
    logic      tile_last; // 触发冲刷
} deq_tag_t;           // 13 bit × 5 级 = 65 FF/核，全片 520 FF
```

alpha_en 位已随裁决 D4 删除，标签从 14 bit 降为 13 bit。

前置条件	是否成立	理由
权重能在 1 拍内切换	成立	加法树的乘法在第 1 级，双缓冲寄存器全局 swap 只需 1 拍
输入 FIFO 顺序天然匹配	成立	MCU / GCore 按静态调度依次压入，消费顺序与压入顺序一致
相邻拍无累加器端口冲突	成立	相邻拍写不同存储，或同一存储的顺序地址，1R1W 足够

完整的写回标签束（跨 11 拍传播，用于 § 12.4 的接口与 § 16.3 的记分板）还包含：

标签字段	最小宽度	用途
epoch	4	拒绝 abort/reset 后的过期响应
opcode / post_op	6 / 5	选择写回与转换
dst	8	寻址本地/GCore/CNET 端点
row	6	suffix 行 0-50
tile_n / tile_k	8 / 8	矩阵坐标
head	3	query head / PCore 身份一致性
blk_id	6	Attention 块 0-54
lane_mask	16	需要时抑制物理尾部写
final_k / last	1 / 1	累加器提交与操作退休

6.4.4 FACC

项	规格	说明
---	----	----

组织	FP32 512b × 51 × 2	6.38 KiB，1R1W，乒乓
为什么必须乒乓	—	N-tile n 的冲刷（51 拍）与 n+1 的首个 K-tile 累加（51 拍）完全重叠
共用范围	全部 GEMM	q/k/v/o/gate/up/down 与 QK^T 共用同一块
例外	P·V	累加器是 OACC 本身，见 § 7.8

6.5 VPU 与 SFU

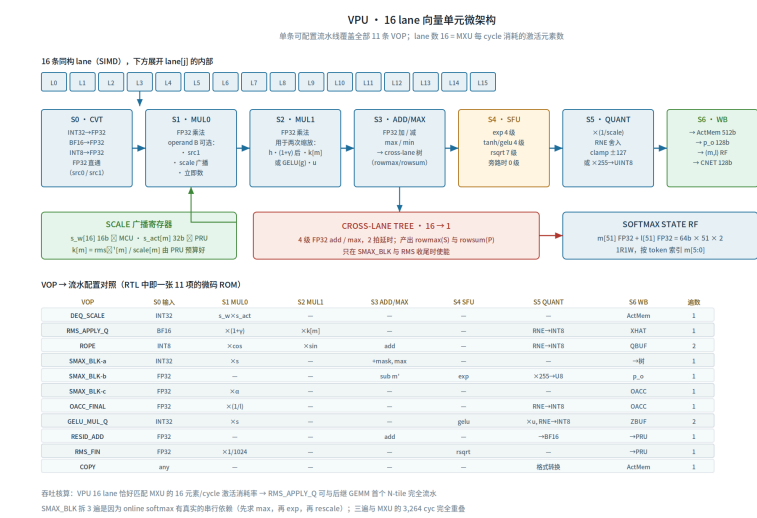


图 6-5 VPU 的 16 lane 微架构与 VOP 流水配置（源：V1.0 图 7）

VPU 是一条 7 级可配置流水线，16 个 lane 恰好等于 MXU 每拍消耗的激活元素数。**乘法器为真 FP32 × FP32（裁决 D5），不得用 BF16 乘法器替代。**

6.5.1 VOP 操作集

VOP	输入	语义	输出 / 周期
VOP_BF16_TO_FP32	16 BF16	精确加宽	16 FP32 / 1 使能拍
VOP_FP32_TO_BF16	16 FP32	RNE ，IEEE NaN 策略	16 BF16 / 流水后 1 拍
VOP_ADD / VOP_MUL / VOP_FMA	FP32	逐 lane；FMA 单次舍入	16 FP32 / 拍
VOP_BLKMAX	16 FP32	跨 lane 求块内最大值 → 计算 E → 写 ScaleScale 表	1 个 E / 拍

VOP_QUANT_MX	16 FP32 + E	$q = \text{RNE}(x \cdot 2^{(133-E)})$, clamp	16 INT8 / 拍
VOP_ROPE	2 遍	复数旋转 (FP32 乘 + FP32 加) → BLKMAX + QUANT	见 § 8.12
VOP_SMAX_ROWMAX	FACC 冲刷流	施加掩码并更新行最大值	折进冲刷, 零额外拍
VOP_SMAX_EXP_Q	SBUF	读 SBUF → exp → 更新 ℓ → 块最大值 → 量化写 PBUF	见 § 7.6
VOP_OACC_SCALE	OACC word, $\alpha[\text{row}]$	逐 lane 真 FP32 乘 α (D4)	一个 512b word/拍; 816 拍全量
VOP_OACC_FINAL (A_FIN)	OACC, ℓ	$O \leftarrow OACC \times \text{recip}(\ell) \rightarrow \text{BLKMAX} \rightarrow \text{QUANT}$	816 拍
VOP_GELU_MUL_Q	G, U	$z = \text{GELU}(g) \odot u \rightarrow \text{BLKMAX} + \text{QUANT}$	2 遍
VOP_RMS_APPLY	BF16 行片段, inv_σ	$\text{FP32}(x) \cdot \text{inv}_\sigma$ 后进入 MX 编码	16 值/拍
VOP_EULER	x, v, dt	$\text{FP32}(x) + dt \cdot \text{FP32}(v) \rightarrow \text{BF16}$	16 FP32 后 BF16 落盘
VOP_FACC_FLUSH	FACC	冲刷 FACC, 可选乘 inv_σ	51 拍/tile
VOP_COPY	—	搬运与格式转换	—

6.5.2 VOP_OACC_SCALE 的资源仲裁（规范性）

VOP_OACC_SCALE 读一个 512 bit OACC word（16 个 FP32），把全部 lane 乘以一个逐行的 FP32 α ，再按同样的固定地址序写回。对 [51,256] 共 816 个 word，因此需要 **816 个使能周期**，其间独占 OACC 的 1R1W 端口。

资源冲突	解决
OACC_SCALE vs PV	永不重叠 ；两者都要 OACC 端口
OACC_SCALE vs QK	允许；QK 写 SBUF/FACC，不访问 OACC
OACC_SCALE vs EXP	允许；EXP 用 SBUF/PBUF/SFU
OACC_SCALE vs 全局停顿	读、乘、写三级原子冻结

6.5.3 SFU

6.6 GCore 微架构

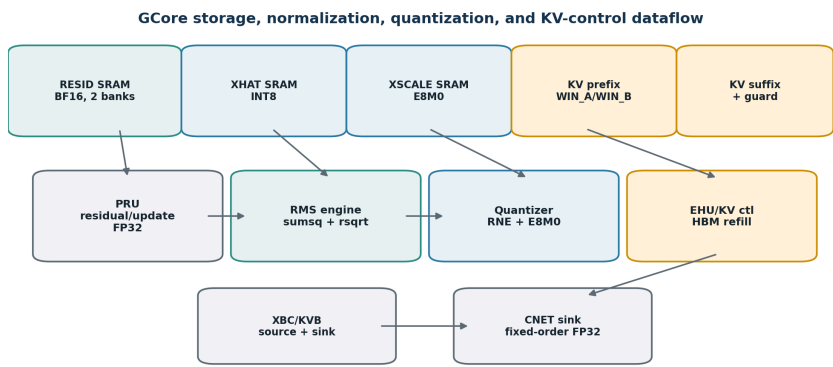


图 6-7 GCore 数据流、存储归属与 KV 控制（源：V1.2 Fig.6-3）

单元	lane / 精度	功能
PRU	4 个 ingest lane, FP32 累加	CNET 归约汇聚、残差加、平方和收集
RMS/Quant 通路	16 BF16 输入, FP32 乘/比较	Pass B 归一化与 MXINT8-B16 转换
EHU	FP32 标量/向量	time embedding LUT、Euler 更新、小状态投影
KV 控制器	128 bit 数据 + Scale 旁带	双窗 prefix 预取、suffix 追加、KVB 发起
错误/遥测	always-live 控制域	粘滞格式/ECC 故障、FIFO 最低水位、停顿原因与拍数

6.6.1 PRU 的六级流水



图 6-8 GCore 中 PRU 的数据通路与增量归约（源：V1.0 图 9）

级	名称	动作	资源
P0	STAGE	聚合 CNET 输出 → 16 通道 = 恰好一个 MXINT8 块	16 × 32b 寄存器

P1	RESID RD	读 RESID[m×64 + n] (256b = 16 × BF16) → FP32	SRAM 读口 + 16 个转换器
P2	ADD	$h' = h + Y$	4 × FP32 加法器
P3	RESID WB	FP32 → BF16 (RNE , 裁决 D15), 聚合成 256b 写回	SRAM 写口
P4a	Σx^2 累加	$h'^2 \rightarrow 4 \text{ 输入加法树} \rightarrow sq[m] +=$	4 乘 + 2 级树 + 51 × 32b
P4b	块最大值	$\ h'\ \rightarrow 4 \text{ 输入 max 树} \rightarrow E_a[m,n]$	4 abs + 2 级树 + 8 位递增器

P4a 与 P4b 并行，都挂在 P3 的输出上。由于 γ 已折进权重（§ 8.11），P4b 直接取 $|h'|$ 的最大值，不需要额外乘法。

两处必须遵守的精度约束

Σx^2 必须用 FP32 累加。 1024 项同号累加下，BF16 的误差为 $\sqrt{1024 \times 1.95e-3} = 6.2\%$ ，会让 $inv_σ$ 有 3.1% 的误差——而 $inv_σ$ 是乘在整行 1024 个数上的公共系数，是 INT8 台阶（0.79%）的 4 倍。FP32 下误差 1.5e-5，可忽略。

P4a 与 P4b 必须使用已舍入到 BF16 的 h' ，而不是加法器输出的 FP32。因为后续量化读的是 RESID 中的 BF16 值；若块最大值来自 FP32 而量化用 BF16，可能越界。RTL 中的顺序必须是「FP32 加 → 舍入到 BF16 → 转回 FP32 → 平方 / 取最大值」。

裁决 D15（派生） V1.0 的 P3 使用**随机舍入**（LFSR）以缓解 18 层残差链的累计偏置，V1.2 § 4.4 则规定”架构舍入模式为 RNE，随机舍入不属于确定性基线”。V2.0 取 **RNE**：确定性与逐位可复现是 I9 与 § 4.8 的硬性要求，随机舍入会使金标向量失效、使 § 16.3 的记分板无法做位比较。LFSR 通路保留在 RESID_SR_EN CSR 后（默认 0，不在一致性范围内），供 M1 量化 18 层累计偏置时作对照实验。

7 Attention 架构（B_kv = 16）

7.1 逻辑序列与物理序列

区域	物理槽位	有效性	RoPE 位置
Prefix	0 – 815	基线全有效；PREFIX_RUNTIME_EN=1 时由 PREFIX_VALID[j] 给出	prefix cache 自带（VLM 侧已旋转）
Suffix	816 – 866	51 个槽位全有效	ROPE_POS_BASE + m，基线 816 + m
Padding	867 – 879	恒无效	忽略；实现可写零

PREFIX_VALID_COUNT 必须等于 popcount(PREFIX_VALID)，否则在接受启动前置配置致命错误。基线两者均为 816。

7.2 为什么必须分块

完整的 S[51, 880] FP32 需要 $51 \times 880 \times 4 \text{ B} = 179.5 \text{ KB/核}$ ，超过整个 PCore 的 128 KB 预算。P 同样大。FlashAttention 用在线 softmax 把 S 与 P 的存储降到一个块的大小：

方案	S 存储 / 核	P 存储 / 核	是否可行
全展开	179.5 KB	44.9 KB	否
B_kv = 64	$12.75 \text{ KB} \times 2$	$3.19 \text{ KB} \times 2$	可行但有 ragged 块
B_kv = 16	$3.19 \text{ KB} \times 2$	$0.80 \text{ KB} \times 2$	本设计

7.3 分块方案：B_kv = TILE_R = 16

本设计取 B_kv = 16，即每个 KV 块恰好是一个 MXU tile。这使不变量 I5 的同构关系再延伸一层：

MXINT8 块 $\equiv$ MXU 的 K-tile $\equiv$ 加法树输入 $\equiv$ FlashAttention 的 KV 块 = 16

段	列数	块	说明
prefix	816	块 0 – 50 (51 块)	816 = 51 $\times$ 16 整除，无 ragged 块
suffix	51 $\rightarrow$ 64	块 51 – 54 (4 块)	补齐到 16 对齐，padding 列由掩码处理
合计	880	55 块，每块 1 个 tile	相对 S = 867 的 padding = 1.50%

QK 拍数 = $55 \times (\text{Nt}=1 \times \text{Kt}=16 \times \text{M}=51) = 44,880$

PV 拍数 = $55 \times (\text{Nt}=16 \times \text{Kt}=1 \times \text{M}=51) = 44,880$

合计 89,760 拍，与 B_kv = 64 完全相同

分块大小不改变计算量，只改变调度粒度与中间存储。

相对 B_kv = 64 的三处结构简化

- ① **55 个块全部等长**。B_kv = 64 时 $816 \div 64 = 12.75$ ，最后一个 prefix 块只有 3 个 tile，需要 $\text{Nt} = (\text{blk} == 12) ? 3 : 4$ 的特例分支。B_kv = 16 时不存在这个分支。
- ② **P 的ScaleScale索引降一维**。B_kv = 64 时每块含 4 个 P Scale块，索引为 $\text{E_p}[m, k_S]$ ；B_kv = 16 时每块恰好一个，索引简化为 $\text{E_p}[m]$ 。
- ③ **SBUF 与 PBUF 显著变小**。S_b 从 [51,64] 变为 [51,16]，乒乓后从 25.5 KiB 降到 6.38 KiB；PBUF + E_p 同比缩小。全核合计省 24.2 KiB/核、全片 194 KiB。

代价（必须记住） PV 段的 Kt = 1，因此 **FACC 不参与 PV**，累加器直接是 OACC，且 OACC 端口占空升至 100%。这是 § 7.8 中 OACC 必须 1R1W 且 OACC_SCALE 与 PV 不得重叠的根源。

7.3.1 B_kv 选择的完整权衡

B_kv = 16 与 B_kv = 64 的权衡			
本设计选 16：结构统一、存储更省，代价是 OACC 端口无余量			
维度	B_kv=16（本设计）	B_kv=64	判定
块数 / 结构	55，全等长 (526+51×16)	14，18.12为310w (ragged)	16 胜：无特例分支
与 TILE_R 的关系	B_kv = TILE_R = 块 = 段 = 16	B_kv=4 × TILE_R	16 胜：自举同构
SBUF（片尾）	6.58 KiB	25.50 KiB	16 胜：省 19.1 KiB
PBUF + E_p（片尾）	1.69 KiB	6.78 KiB	16 胜：省 5.1 KiB
PV 的 Kt	1（FACC 不参与）	4（FACC 参与累加）	—
OACC 端口占空	100%（PV 段全程 RRRR）	25%	64 胜：16 无余量
累加器能耗（PV 段）	2.8 单位	1.78 单位	64 胜：16 多 60%
rowmax 数量	55 × 5 = 275 组	14 × 5 = 70 组	64 胜：~200 组
GERM INDEX 数	110	28	64 胜（但数据流未完全差到）
掩码步长块数	4（18 51-54）	1（18 13）	64 稍胜
α 误差误差	4.8e-4	2.3e-4	均 < INT8 台阶 7.8e-3
延迟前	520,911	520,706	<+0.04%
动作延迟时	94.57 ms (10.57 Hz)	94.54 ms (10.58 Hz)	可忽略

结论：选 16，存储节省 24.2 KiB/核（全片 194 KiB）与结构统一是确定收益；OACC 100% 占空在功能上无问题（该端口本数量 512b 1R1W 配置），只是失去余量，若后续需要让其他单元访问 OACC，应重新评估。

图 7-1 B_kv = 16 与 B_kv = 64 的权衡（源：V1.0 图 17）

维度	B_kv = 16（本设计）	B_kv = 64	判定
块数	55，全等长	14，末块 ragged	16 优
ragged 特例分支	无	需要	16 优
SBUF + PBUF + E_p	8.07 KiB/核	32.3 KiB/核	16 优（省 24.2 KiB/核）
P Scale索引维度	1 维 E_p[m]	2 维 E_p[m,k]	16 优
FACC 参与 PV	否（Kt = 1）	是（Kt = 4）	64 优
OACC 端口占空	100%（PV 段）	25%	64 优
α 事件数	55 次	14 次	64 优
exp 的 SFU 峰值占用	50%	50%	相同
总 MAC	相同	相同	—
裁决	采用 16：存储与控制的简化压过 OACC 端口压力，后者可用 1R1W 化解		

7.4 掩码（裁决 D6）

见 § 3.6 的完整推导。执行侧的规范性要求：

jj = ((blk - 51) << 4) + j

masked = (jj >= 51) OR (m == 0 AND jj >= 1) // 仅 blk >= 51 时非平凡
score = masked ? NEG_INF_FP32 : score

- MASK_EN (AUX.MASK_MODE = 1) 仅在 blk >= 51 的 QK 描述符上置位。
- 掩码必须在 rowmax 之前施加，与 FACC 冲刷同拍完成 (VOP_SMAX_ROWMAX)，零额外周期。
- padding 列 (jj ≥ 51) 与被屏蔽列一律 $-\infty$ ，不得用”置 0 后修正”的做法。
- PREFIX_RUNTIME_EN=1 时，块 0-50 的掩码取自 KVB 广播的 kvb_valid_mask[15:0]。

7.5 在线 softmax 递推

五个算子的形式化定义

下标 b = KV 块, m = token, j = 块内位置或输出通道, i = 归约位置

A	分数计算 • MXU + DEQACC tile (n_S, k_D): psum[m,j] = $\sum_{i=0}^{15} Q_{[m, 16k_D+i]} \cdot K_{[16n_S+j, 16k_D+i]}$ FACC[m,j] += psum[m,j] $\cdot 2^{(E_q[m,k_D] + E_k[16n_S+j, k_D] - 254)}$ k_D = 15 后: S_b[m, 16n_S+j] ← FACC[m,j] $(1/\sqrt{d} = 2^{E_q})$ 已折进 E_q
A'	掩码 • VPU (仅 b = 13) S_b[m,j] ← $-\infty$ 若 j ≥ 51 或 (m = 0 ∧ j ≥ 1) 在写入 SBUF 的同一拍施加，必须早于 ρ_b 的计算
B	在线 softmax 状态 • VPU + SFU ρ_b[m] = max_j S_b[m,j] (随 SBUF 冲刷增量求出) μ_b[m] = max(μ_{b-1}[m], ρ_b[m]) α_b[m] = exp(μ_{b-1}[m] - μ_b[m]) (旧累加值的重缩放因子)
C	指数与块量化 • SFU + VPU P_{[m,j]} = exp(S_b[m,j] - μ_b[m]) ℓ_b[m] = ℓ_{b-1}[m] $\cdot \alpha_b[m] + \sum_j P_{[m,j]}$ v_b[m,k_S] = max_{j ∈ [16k_S, 16k_S+16]} P_{[m,j]} E_p[m,k_S] = log(v_b[m,k_S] / 127) P_{[m,j]} = RNE(P_{[m,j]} / 2^{E_p[m,k_S]})
D	输出累加 • MXU + DEQACC tile (n_D, k_S): psum[m,j] = $\sum_{i=0}^{15} P_{[m, 16k_S+i]} \cdot V_{[16k_S+i, 16n_D+j]}$ FACC[m,j] += psum[m,j] $\cdot 2^{(E_p[m,k_S] + E_v[16n_D+j, k_S] - 254)}$ k_S = 3 后: O[m, 16n_D+j] ← O[m, 16n_D+j] $\cdot \alpha_b[m] + FACC[m,j]$
E	归一化 • VPU + SFU (全部 14 块之后) O_final[m,c] = O[m,c] $\cdot \text{recip}(\ell_{13}[m])$

E8M0 的指数计算：给定 FP32 的 x，求最小 E 使 $x/2^E \leq 127$
E = ex - 133 若尾数 < 127/64 = 1.98431；否则 E = ex - 132 (一个 8 位减法 + 一个尾数比较器)
初值：μ_{[-1]}[m] = -1e30 (不用 -∞，使 α_0 走 exp 的 x < -20 → 0 边界，避免 NaN)；ℓ_{[-1]}[m] = 0；O = 0

图 7-2 Attention 五个算子的形式化定义（源：V1.0 图 12）

符号	含义	格式与规模
μ_b[m]	到块 b 为止的行最大值	FP32 × 51
ℓ_b[m]	到块 b 为止的分母	FP32 × 51
α_b[m]	旧累加值的重缩放因子	FP32 × 51 × 2 (必须乒乓)
ρ_b[m]	块 b 内的行最大值 (16 列上求得)	FP32 × 51
v_b[m]	P 的块内最大值	FP32 × 51

```

rho_b[m] = max_j S_b[m,j] over valid j in block b
mu_new[m] = max( mu_old[m], rho_b[m] )
alpha_b[m] = exp( mu_old[m] - mu_new[m] )    # l_old == 0 时强制为 0
p_b[m,j] = exp( S_b[m,j] - mu_new[m] ) for valid j; else 0
l_new[m] = alpha_b[m] * l_old[m] + sum_j p_b[m,j]
O_new[m,:] = alpha_b[m] * O_old[m,:] + p_b[m,:] @ V_b
O_final = O / l

```

分数缩放 $1/\sqrt{256} = 2^{-4}$ 通过 $\text{AUX.EXP_FOLD} = -4$ 折入（§ 4.6）。掩码在 rowmax 之前施加。没有有效 key 的块不改变 μ 、 l 、 O ，且不得求值 $\exp(-\text{Inf} - (-\text{Inf}))$ （§ 3.6.2）。

7.5.1 正确性论证

递推的正确性可归纳证明。设处理完块 b 后：

```

O_b[m,c] = sum_{s <= 块b末} exp( S[m,s] - mu_b[m] ) * V[s,c]
l_b[m] = sum_{s <= 块b末} exp( S[m,s] - mu_b[m] )

```

处理块 b+1 时最大值从 μ_b 更新到 μ_{b+1} ，所有旧项的指数都要减去差值：

$$\exp(S - \mu_{b+1}) = \exp(S - \mu_b) * \exp(\mu_b - \mu_{b+1}) = \exp(S - \mu_b) * \alpha_{b+1}$$

因此 $O \leftarrow O \cdot \alpha + P_{b+1} V_{b+1}$ 与 $l \leftarrow l \cdot \alpha + \sum P_{b+1}$ 保持归纳假设，最终 O/l 即为精确 softmax 的结果。■

数值上还需三条：

1. 所有 exp 的输入恒 ≤ 0 （因为减去了 $\mu_{\text{new}} \geq$ 该块任意分数），故 $\exp \in (0, 1]$ ，无上溢。
2. $\alpha \in (0, 1]$ ，54 次连乘的相对误差在 FP32 下为 $4.4e-4$ （§ 4.15），BF16 下不可接受——这是裁决 D5 的直接依据。
3. l 恒为正且有限：至少 state token 自身与 action token 的自注意力项贡献 $\exp(0) = 1$ 。A_FIN 的 recip 输入因此有下界。

7.6 跨块软件流水

一个块内的四个动作严格串行：算分数 → 更新最大值 → 算指数 → 累加输出。其中”算指数”占用 SFU 而 MXU 空闲，若逐块串行执行，MXU 每块要空转。

关键观察：QK(b+1) 不依赖块 b 的任何结果——它只需要 Q（全程不变）与 K(b+1)。因此可以把 QK 提前，让 exp 与 PV 重叠。



图 7-3 跨块软件流水的完整时间线（源：V1.0 图 15）

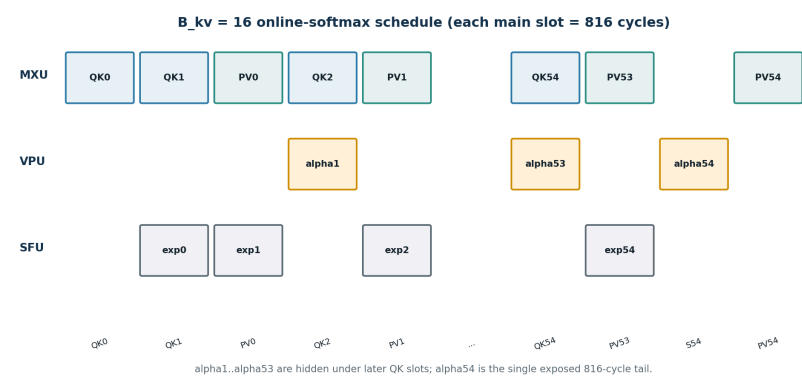


图 7-4 α 重缩放是藏在 QK 下的 VPU 操作，仅末块暴露（源：V1.2 Fig.7-1）

```
// 前导
QK(0) // 816 拍
QK(1) || EXP(0) // exp(0) 需 408 拍（2 lane），占 50%
PV(0) || EXP(1) // OACC 为零，alpha0 缩放省略

// 稳态循环 b = 1..53（REP = 53）
for b in 1..53:
    QK(b+1) || VOP_OACC_SCALE(alpha[b]) // 816 拍，α 完全隐藏
    PV(b) || EXP(b+1) // 816 拍

// 收尾
VOP_OACC_SCALE(alpha[54]) // ★ 暴露 816 拍（裁决 D4）
PV(54) || A_FIN 尾随写回
```

exp	窗口	拍数	需求	2 lane 占用
exp(0)	QK(1)	816	816 个	50%（最紧的一处）
exp(1…53)	PV(b-1) + QK(b+1)	1,632	816 个	25%
exp(54)	PV(53)	816	816 个	50%

把 α 的 exp 一并计入：Attention 段共需 $55 \times 816 + 55 \times 51 = 47,685$ 次 exp，摊在 89,760 拍上是 0.531 次/拍，2 lane 的占用率为 **27%**。

7.6.1 α_{54} 尾的成因与账目（裁决 D4）

α_b 的重缩放必须在 PV(b) 之前完成（否则旧的 OACC 会被新块的贡献污染）。对 $b = 1 \cdots 53$ ，OACC_SCALE(α_b) 藏在 QK(b+1) 下——两者不争 OACC 端口。但 α_{54} 之后已无 QK 可藏（ $b+1 = 55$ 越界），因此它是一个**暴露的 816 拍事件**。

方案	α 归属	暴露拍数/层	代价
V1.0	DEQACC L3 条件乘，折进 OACC RMW	0	DEQACC 内引入 16 个乘法器； α 精度受 DEQACC 通路限制
V2.0 (D4)	VPU VOP_OACC_SCALE，真 FP32	816	每层 816 拍 = 每步 14,688 拍 = 层周期的 0.157%

裁决理由： α 是 54 次连乘的关键项，必须是可审计的真 FP32 运算；把它藏进 DEQACC 会让精度契约无法在单元级验证。0.157% 的周期代价可接受。

7.7 P 的块量化：Attention 精度的关键

若 P 使用固定Scale（如 1/127）：

每元素 $\sigma = (1/127)/\sqrt{12} = 0.00227$

867 项累加 $\sigma = \sqrt{867} * 0.00227 = 0.0669 * \sigma_V$

均匀注意力下 $\sigma_O = \sigma_V * \sqrt{\sum P^2} = \sigma_V/\sqrt{867} = 0.0340 * \sigma_V$

-> 相对误差 ~ 200%

注意力越分散误差越大——大量小 P 值被直接压成 0。改用**沿 seq 的ScaleScale**后：

$\sigma_{err} = 1.443/(127*\sqrt{12}) * \nu_b = 0.00328 * \nu_b$

16 项累加 $\sigma = 0.0131 * \nu_b * \sigma_V$

-> 相对误差 ~ 0.60%

这是采用 MXINT8 最实质的收益。连带好处是 P 只用 [0, 127] 正半区，不需要偏移二进制方案及其修正项。B_kv = 16 时每块恰好一个 P Scale 块，索引简化为 E_p[m]，比 B_kv = 64 少一维。

注意 P 的块方向是**沿 seq (key)**，与 V 一致（§ 4.10）。这是 PV 段 Kt = 1 的直接原因：一个 P 块恰好是一次 16 项归约的全部输入。

7.8 缓冲与端口

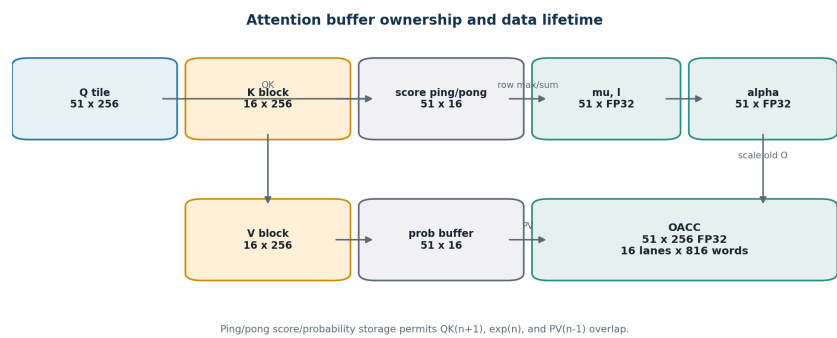


图 7-5 Attention 缓冲、生产/消费相位与 OACC 归属（源：V1.2 Fig.7-2）

存储	格式 / 形状	端口	理由
SBUF 乒乓	$2 \times [51,16]$ FP32	各 1RW	QK 填充的同时较早的块正被 EXP 消费
PBUF 乒乓	$2 \times [51,16]$ INT8 + E8M0	各 1R1W	EXP 写一代的同时 PV 读另一代
OACC	$[51,256]$ FP32	512 bit 1R1W	PV 与 OACC_SCALE 都要全速率 RMW，但 永不重叠
$\mu / \ell / \alpha$ RF	每状态 51 个 FP32	1R1W	固定行序； α 保留两代
Q-Memory	$[51,256]$ MXINT8-B16	1RW	先存 Q，A_FIN 后复用为归一化的 Attention 输出

跨块状态的生命周期				
八项状态，三种生存期：块内、跨块、跨层				
状态	容量	生存期	读写时机	备注
FACC[31,64]×2	6.38 KB	块内 tile 量	QK 读 k 累加 / PV 读 k 累加	无冲相解耦
SBUF[31,64]×2	25.5 KB	跨 1 块	QK(b) 写 → exp(b) 读	ping-pong，读后即写可
PBUF[31,64]×2	6.38 KB	跨 1 块	exp(b) 写 → PV(b) 读	ping-pong
$\ell_p[31,64] \times 2$	0.80 KB	跨 1 块	随 PBUF	ping-pong
$w[51] \times 2$	0.40 KB	跨 1 块	QK(b) 读 → PV(b-1) 后读 PV(b) 用	必须乒乓；QK(b+1) 会覆盖
$m_mem[51]$	0.20 KB	跨全部 14 块	每读 QK 更新，每读 exp 读	读出即可（严格读后使用）
$l_mem[51]$	0.20 KB	跨全部 14 块	每读 exp 更新	单份
OACC[31,256]	51 KB	跨全部 14 块	每读 PV 读时 RMW	单份，A_FIN 后释放

合计 attention 相关 98.0 KB/层（占层内 351 KB 的 28%）

三条最窄输出链的生命周期约束

① w 必须乒乓：物理流水下 QK(b+1) 在 PV(b) 之前执行，会在 $w(b)$ 被阅读之前读出 $w(b+1)$ 。

② m_mem 单侧即可：exp(b) 使用前 m_mem 必须已含块 b，而 QK(b) 产值早于 exp(b)，能天然保证。

③ l_mem 的更新依赖到 exp(b) 时刻（而非 PV(b)），从而不必保存 rowsum_b —— 省一倍 51×4 B 寄存器。

SBUF 从单缓冲修改为乒乓，是全流程主导（QK(0), QK(1) 首条链）唯一的额外代价。

图 7-6 Attention 相关状态的生存期与乒乓需求（源：V1.0 图 23）

乒乓判据（§ 10.3 给出通式）：一个缓冲需要乒乓，当且仅当其生产窗口与消费窗口在时间上重叠。
SBUF、PBUF、FACC、 α 满足；OACC 不满足（生产与消费严格分相），故单体 1R1W 足够。

7.9 逐块执行与无数学缺口

对行 m 与块 b，物理 key 索引为 $j = 16b + n$ ， $n = 0 \cdots 15$ 。块有效位 $vb[n] = \text{valid}(m, 16b+n)$ 。

for m = 0..50:

for n = 0..15:

```
j = 16*b + n
score[n] = dot(Q[m,:], K[j,:]) * 2^-4
masked[n] = score[n] if valid(m,j) else -INF
if OR(valid[0:15]) == 0:
    preserve mu[m], l[m], OACC[m,:]; continue    # 空块规则
rho  = max(masked[0:15])
mu_new = max(mu_old, rho)
alpha = exp(mu_old - mu_new) if l_old != 0 else 0
p[n]  = exp(masked[n] - mu_new) if valid[n] else 0
l_new = alpha*l_old + sum(p[0:15])
O_new[:] = alpha*O_old[:] + sum_n( p[n] * V[j,:] )
```

l_old == 0 谓词定义了初始化并避免求值 exp(-Inf - finite)。对第一个有效块，α 强制为 0，因此不需要读 O_old。基线中块 0 恒有效；PREFIX_RUNTIME_EN=1 且 prefix 全无效时，保持规则一直重复到块 51（首个 suffix 块，恒有效）。

7.10 块内逐单元时间线

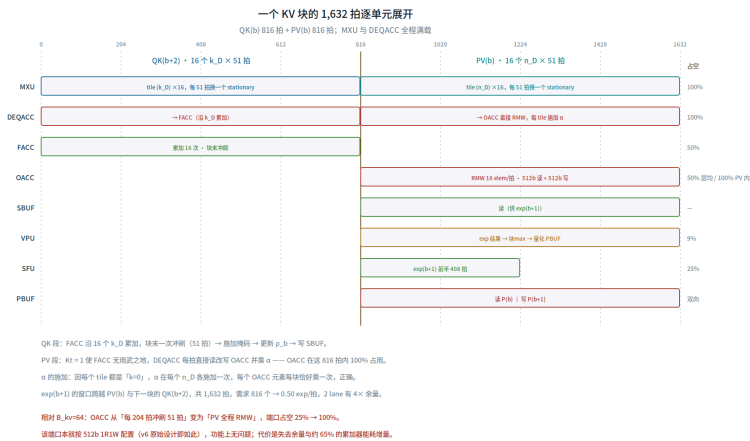


图 7-7 一个 KV 块的 1,632 拍逐单元展开（源：V1.0 图 16，α 归属已按 D4 更新）

单元	QK(b+2) 段（前 816 拍）	PV(b) 段（后 816 拍）	占空	余量
MXU	tile (k_D) × 16	tile (n_D) × 16	100%	瓶颈
DEQACC	→ FACC（沿 k_D 累加）	→ OACC 直接 RMW（不施加 α）	100%	与 MXU 同速率
FACC	累加 16 次 · 块末冲刷	不参与（Kt = 1）	50%	2×
OACC	被 OACC_SCALE(α_b) 占用	RMW 16 elem/拍	两段均 100%	无余量
SBUF	块末写 51 拍	连续读（供 exp(b+1)）	读写分处两半程	单端口足够

VPU	OACC_SCALE 816 拍 + 掩码/rowmax 51 拍	块最大值 + 量化	QK 段 ~100%	无余量
SFU	空闲	exp(b+1) 前半 408 拍	25%	4× (2 lane)
PBUF	写 P(b+1) 续	读 P(b) 写 P(b+1)	双向	必须 1R1W + 乒乓
KVB	K tile × 16	V tile × 16	33.3%	3×

裁决 D4 使 VPU 从 9% 占空升到接近 100%（QK 段）。这是 D5 要求真 FP32 乘法器时面积上升的原因，也是 § 11.5 把 core 面积下限从 5.1 mm² 抬到 5.5 mm² 的依据。

7.11 A_FIN: Attention 输出的归一化与量化

A_FIN 把完成的 OACC 逐行除以 ℓ ，并把 [51,256] 输出量化进 Q-Memory，随后作为 O 输入送给输出投影。

```

recip_l[m] = reciprocal( l[m] )           # SFU，51 次
for each 512b OACC word (816 个):
    o_fp32[0:15] = OACC_word * recip_l[m]   # VPU FP32 乘
    nu          = blkmax( o_fp32[0:15] )    # VOP_BLKMAX
    E_o[m,blk]  = encode_exp( nu )          # § 4.4.1
    o_q[0:15]   = quant( o_fp32, E_o )      # VOP_QUANT_MX

```

项	值
周期	816 拍（与 PV(54) 尾随重叠）
SFU 需求	51 次 reciprocal，7 级流水，可忽略
输出去向	Q-Memory 原地复用为 $\hat{O}$ + E_o
端口	OACC 读 + Q-Memory 写，无冲突

Q-Memory 复用的相位约束：Q 写、Attention 读、A_FIN 覆写三者**相位互斥**，由 § 9.2 的计分板保证。RTL 必须断言 A_FIN 的写地址不早于该行 Q 的最后一次读。

7.12 Attention 控制状态

状态	占用资源	完成谓词	下一状态
A_CLEAR	$\mu/\ell/\alpha$ RF、OACC valid	51 个状态项初始化；OACC valid 清零	A_QK0

A_QK	MXU、Q、KVB、SBUF 写	816 次分数写提交	A_EXP 或重叠的后继
A_EXP	SBUF 读、SFU、PBUF 写、标量 RF	51 行归约与概率块提交	A_PV 可发起
A_SCALE	VPU、 α RF、OACC RMW	816 次 OACC 写提交	A_PV 可发起
A_PV	MXU/DEQACC、PBUF/V、OACC RMW	816 次 OACC 更新提交	下一块或 A_SCALE54
A_SCALE54	VPU、OACC RMW	816 次写提交	A_FIN
A_FIN	OACC/ ℓ 、reciprocal SFU、量化器、Q-Memory	816 个输出字与 Scale 有效	A_DONE
A_DONE	无数据端口	计分板退休被接受	IDLE

7.13 强制断言（规范性）

- 对每个 query，有效 key 数 = popcount(PREFIX_VALID) + 51；padding 867–879 永不贡献 rowmax、 ℓ 或 PV。
- PREFIX_VALID_COUNT == popcount(PREFIX_VALID)，否则启动前置配置致命错误。
- 每个 QK / EXP / SCALE / PV 请求的 BLK_ID < 55。
- PBUF 的第 b 代恰由 EXP(b) 写一次、由 PV(b) 读一次。
- OACC_SCALE(b) 仅在 α (b) 有效之后、PV(b) 之前发起。
- OACC_SCALE 与 PV 永不在同一周期占用 OACC 端口。**
- 空块不产生任何 SFU 请求，且 $\mu/\ell/O$ 保持**位相同**。
- A_FIN 的 recip 输入 $\ell[m]$ 对全部 51 个 suffix 行为有限正数。
- MASK_EN 与 BLK_ID >= 51 等价（PREFIX_RUNTIME_EN=0 时）。
- state 行（m = 0）在块 52、53 的全部 16 列、块 51 的第 1–15 列、块 54 的全部列上，其 p 值必须**恒为 0**（裁决 D6 的可观测判据）。

7.14 三处零成本融合

做法	否则的代价	收益
rowmax 折进 SBUF 冲刷（与写同拍）	额外遍历 SBUF 一次：3,264 拍/块	零额外周期
ℓ 更新折进 exp（而非 PV）	需保存 rowsum_b 到 PV 时刻，多 51×4 B 与一个依赖	省一组寄存器

V1.0 曾把” α 折进 OACC 冲刷” 列为第一处零成本融合（省 11,424 拍/层）。裁决 D4 撤销了这一项，代价已在 § 7.6.1 记账。

8 逐算子执行流程

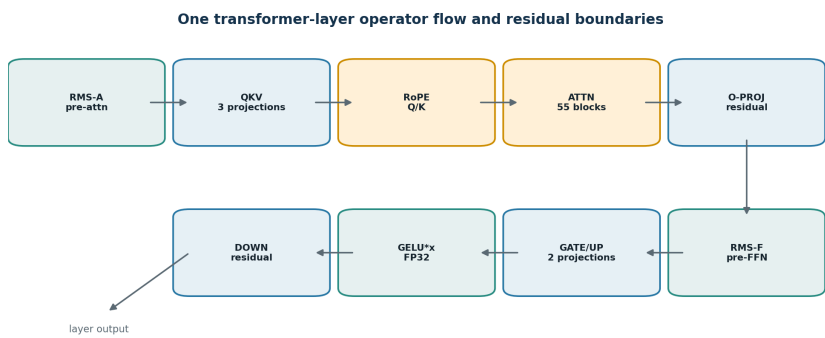


图 8-1 完整的 transformer 层算子流与残差边界（源：V1.2 Fig.8-1）

本章按执行顺序给出单层内每个算子的原始定义、并行切分、硬件映射与时序。每个算子的小节结构一致：原始公式 → 切分与映射 → 执行流程 → 关键实现细节。

8.0 单层程序总览

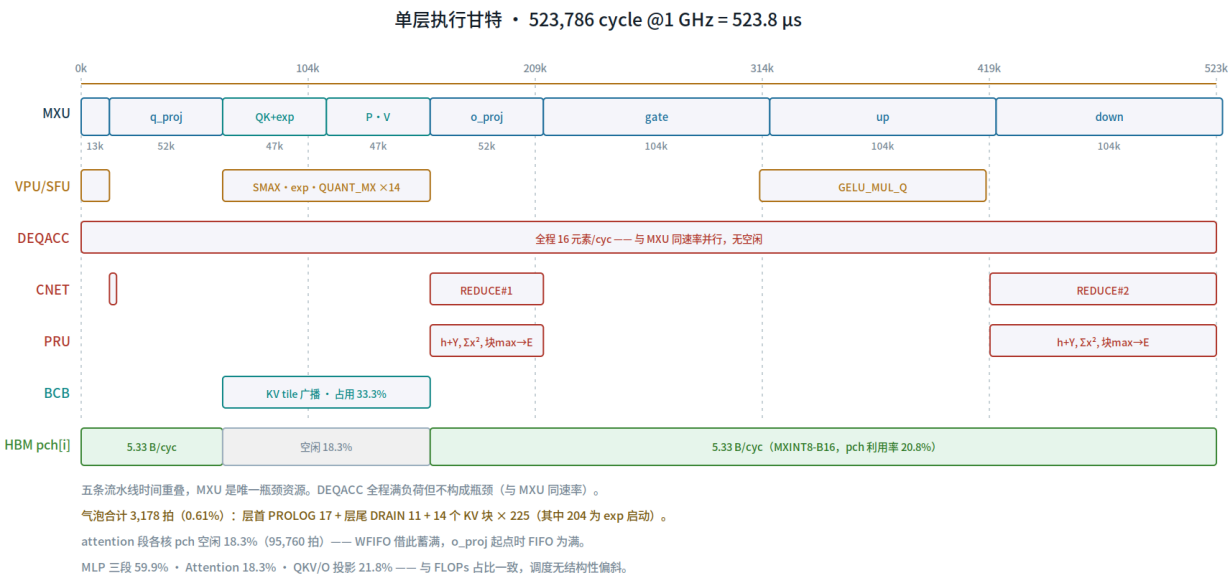


图 8-2 单层执行甘特图：五条流水线的时间重叠（源：V1.0 图 10）

#	算子	执行位置	Kt	Nt	拍数	输出

②	RMS_a Pass B (input norm)	GCore	—	—	3,264	XHAT + E_a
①	kv_proj + RoPE(K) + 量化	PCore	64	4	13,056	K_i/V_i[51,32]
②	KV GATHER	CNET → GCore	—	—	3,264	GCore KV suffix
③	q_proj + RoPE(Q) + 量化	PCore	64	16	52,224	Q ₁ [1,256]
④	Attention (55 块)	PCore	—	—	89,760 + 275	OACC[51,256]
⑤	α_{54} 暴露尾	VPU	—	—	816	OACC
⑥	A_FIN	VPU/SFU	—	—	(与 PV(54) 重叠)	$\hat{O}$ + E_o
⑦	o_proj + REDUCE + 残差	PCore + CNET + GCore	16	64	52,224	RESID
⑧	RMS_f Pass B (post-attn norm)	GCore	—	—	3,264	XHAT + E_a
⑨	gate_proj + up_proj + GELU⊙	PCore	64	32	208,896	$\hat{Z}$ + E_z
⑩	down_proj + REDUCE + 残差	PCore + CNET + GCore	32	64	104,448	RESID
—	层首 PROLOG + 层尾 DRAIN	—	—	—	28	—

层内 MXU 调度合计 **520,608 拍**；加上 PROLOG/DRAIN 28 拍、rowmax 保守余量 275 拍、 α_{54} 尾 816 拍，得 **521,727 拍/层**（不含两次 RMS Pass B，后者统一在 § 11.1 按 37 次/步计入）。

8.1 RMS_ApplyQ（输入归一化）

8.1.1 原始定义

$\sigma[m] = \sqrt{(1/1024) \cdot \sum_c h[m,c]^2 + \epsilon)}$
 $\hat{X}[m,c] = (h[m,c] / \sigma[m]) \cdot (1 + \gamma[c])$
 γ 已按 § 8.11 离线折进后继权重，因此硬件只算 $h \cdot \text{inv_sigma}$ 。

8.1.2 两遍实现（裁决 D3）

遍	何时执行	内容	暴露拍数

Pass A	与前一算子的 CNET 残差写融合	$sq[m] += h'[m,c]^2$ (FP32) , 随 PRU 的 P4a 增量完成	0
P5	Pass A 结束后	$inv_sigma[m] = rsqrt(sq[m] * 2^{-10} + eps)$	26 拍 (SFU, 可忽略)
Pass B	P5 之后	读 BF16 RESID $\rightarrow$ FP32 乘 $inv_sigma \rightarrow$ BLKMAX $\rightarrow$ 求 $E_a \rightarrow$ 量化写 XHAT	3,264

```
// Pass B, GCore 内执行, 8 个 PCore 不参与
for m in 0..50:
  for blk in 0..63:
    x_fp32[0:15] = FP32( RESID[m, blk*16 : blk*16+16] ) * inv_sigma[m]
    E_a[m,blk]  = encode_exp( blkmax(|x_fp32|) )    // § 4.4.1, ceil 型
    XHAT[m,blk] = quant( x_fp32, E_a[m,blk] )      // 16 码/拍
// 3,264 = 51 * 64 拍

÷1024 是指数字段减 10, 不是除法器。
```

8.1.3 执行速率

阶段	单元	速率	一个 n-tile 的拍数	占空
P0–P4 (Pass A)	PRU 4 lane	4 元素/拍	204	25%
P5	SFU rsqrt	2 lane	26 (每次归一化一次)	可忽略
Pass B	RMS/QUANT 16 lane	16 元素/拍	3,264 (整行)	100% (暴露)

8.2 KV_Proj 与 RoPE(K)

8.2.1 原始定义

```
K = X . W_K , V = X . W_V
X : [51,1024] MXINT8-B16 (GCore XHAT, 经 XBC 广播)
W_K : [1024,256] MXINT8-B16
W_V : [1024,256] MXINT8-B16
K : [51,256] -> RoPE -> 量化 (沿 head_dim 分块)
V : [51,256] ->    量化 (沿 seq 分块)
```

8.2.2 切分：8 核各持 K 与 V 的 32 列

K 与 V 各 256 列, 沿 N 维八等分, 每核持有 W_K 的 32 列与 W_V 的 32 列。这样 8 个核的权重量与执行时间完全一致, 不需要为不同的核准备两套段表。

RoPE 的 split-half 约束：RoPE 把通道 j 与 $j+128$ 配对旋转，因此一个核必须同时持有配对的两半，否则旋转需要跨核通信。K 的列映射因此是**两段式**的：

core	W_K 的列	W_V 的列
0	[0,16) $\cup$ [128,144)	[0,32)
1	[16,32) $\cup$ [144,160)	[32,64)
2	[32,48) $\cup$ [160,176)	[64,96)
...	...	...
7	[112,128) $\cup$ [240,256)	[224,256)

通式：core i 持 $W_K[:, 16i:16i+16] \cup W_K[:, 128+16i:128+16i+16]$ 。V 不参与 RoPE，因此其列切分是简单的连续区间。Q 的切分按 head，天生满足配对条件（每核一整个 head 的 256 列）。

8.2.3 执行流程

1. X 经 XBC 从 GCore 广播至 8 个 PCore（256b/拍）
2. 各核从 WFIFO 取自己的 W_K / W_V tile，MXU 采用 weight-stationary 数据流
3. DEQACC 逐 tile 反量化累加进 FACC
4. K 的结果经 VPU 执行 RoPE（FP32 乘 + FP32 加）
5. K 沿 head_dim 求块最大值 -> E_k -> 量化
V 沿 seq 求块最大值 -> E_v -> 量化
6. KV tile 经 CNET 的 GATHER 模式写入 GCore 的 KV-Memory suffix 库

拍数 = $K_t \times N_t \times M = 64 \times 4 \times 51 = \mathbf{13,056}$ 。其中 $N_t = 4$ 是因为每核产出 K 的 32 列加 V 的 32 列共 64 列，即 4 个 16 宽的 tile。

8.2.4 V 沿 seq 量化需要一个暂存缓冲

V 的块是” 16 个连续 seq 位置 $\times$ 1 个通道”，但 kv_proj 的输出是逐 token 产生的（每拍一个 token 的 32 个通道）。因此求块最大值需要先缓存 16 个 token 的结果：

V tile buffer = [16 token, 32 channel] FP32 = 2 KiB / 核

流程：累积 16 个 token -> 逐通道求最大值 -> 计算 E_v -> 量化 -> 送 CNET GATHER

另外，51 个 suffix token 需补齐到 64（4 个块），padding 行必须由 GCore 侧零填充，避免 Inf/NaN 传播。

8.3 Q_Proj 与 RoPE(Q)

$Q = X \cdot W_Q$ ， $W_Q: [1024, 2048] = [1024, 8, 256]$

core i 持 $W_Q[:, 256i:256i+256]$

$K_t = 64, N_t = 16$, 拍数 = $64 * 16 * 51 = 52,224$

流程与 § 8.2.3 相同，差别在：

- 输出留在本核 Q-Memory，**不经 CNET**。
- RoPE 的配对天然在核内（一个完整 head）。
- 量化沿 head_dim 分块（16 个块/行）。
- $1/\sqrt{d_h} = 2^{-4}$ **不在此处折叠**，而在 QK 的 AUX.EXP_FOLD 中施加（§ 4.6）。

8.4 Attention

见 § 7 全章。层内拍数 89,760（QK+PV）+ 275（rowmax 保守余量）+ 816（ α_{54} 尾）。

8.5 A_FIN

见 § 7.11。

8.6 O_Proj + 残差归约 + RMSNorm

8.6.1 切分

$Y = O \cdot W_O, O : [51, 2048], W_O : [2048, 1024]$

O 沿 K 维分布在 8 个 PCore 的 Q-Memory 中（每核 [51, 256]），因此 o_proj 只能按 **K 维（行）** 切分：

core i 持 $W_O[256i : 256i + 256, 0 : 1024]$

$K_t = 256 / 16 = 16, N_t = 1024 / 16 = 64$

拍数 = $16 * 64 * 51 = 52,224$

每核输出 [51, 1024] 的部分和，8 核相加才是最终结果

8.6.2 五级流水



图 8-3 五级流水与数据形态

级	单元	输入	输出	速率	占空
①	VPU + SFU （A_FIN）	OACC FP32	Q-Memory INT8 + E_o	16 元素/拍	与 PV(54) 重叠
②	MXU + DEQACC	Q-Memory + WFIFO	FACC FP32[51,16]	256 MAC/拍	100%
③	CNET REDUCE	$8 \times \mathbf{512b}$ FP32	512b 广播	16 元素/拍	12.5%
④	PRU 4 lane （GCore）	CNET + RESID	RESID + E_a + sq	4 元素/拍	25%
⑤	RMS/QUANT 16 lane （GCore）	RESID + inv_σ	XHAT	16 元素/拍	Pass B 独立暴露

8.6.3 五级在一个 n-tile 窗口内的错位（按 D9 重算）

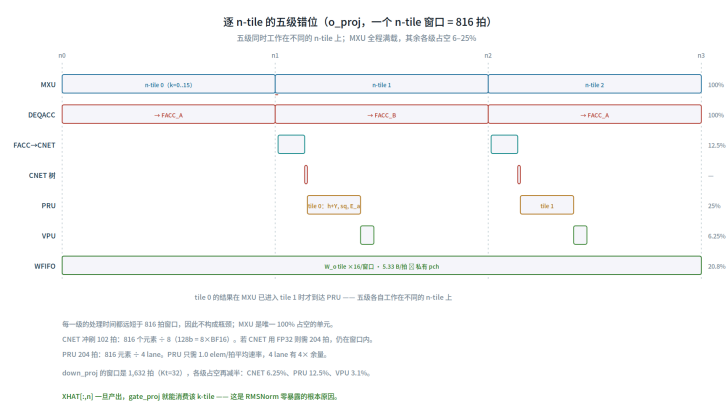


图 8-4 五级在一个 n-tile 窗口（816 拍）内的错位（源：V1.0 图 19，CNET 拍数已按 512b 更新）

相对拍	单元	动作
0 – 815	MXU	tile (n, k = 0…15), 每 51 拍换一个 stationary
11 – 826	DEQACC	→ FACC, 滞后 11 拍 (PIPE_DRAIN)
827 – 877	FACC → CNET	冲刷 51 拍 (816 元素 ÷ 16, FP32 传输, 512b)
878 – 887	CNET 加法树	3 级 FP32 加法, 固定 10 拍
888 – 1091	PRU	204 拍: h+Y、sq +=、块最大值 → E_a[:,n]

8.6.4 尾部延时账目（按 D3 + D9 重算）



图 8-5 尾部延时与 gate_proj 的时间余量（源：V1.0 图 20，数值已按 V2.0 重算）

增量	环节	累计
+11	DEQACC 排空	11
+51	FACC → CNET 冲刷（FP32，512b）	62
+10	CNET 加法树	72
+204	PRU 处理 816 元素	276
+26	PRU P5：rsqrt × 51	302
+3,264	RMS Pass B（暴露，D3）	3,566

需求	截止时刻	实际就绪	余量
XHAT[:, 0] 可被 gate_proj 读取	gate_proj 起点	3,566 拍	−3,566（负）
inv_sigma[m] 可用	Pass B 起点	302 拍	+2,962

结论（与 V1.0 不同）：裁决 D3 之后，RMS Pass B **不再能被隐藏**。它是一个 3,264 拍的独立暴露事件，必须串在 o_proj 与 gate_proj 之间。这正是 § 11.1 中 120,768 拍/步的来源。

D9 把 CNET 冲刷从 204 拍压到 51 拍，回收了 153 拍；这部分收益体现为 Pass B 之前的准备链从 455 拍降到 302 拍，使 Pass B 可以更早开始，但不改变它必须暴露的事实。

这条链依赖一个关键对齐：o_proj 的输出 tile n 恰好就是 RMSNorm 的通道块 b = n（都是 16 个连续通道），使 PRU 处理完一个 tile 就能立即产出该块的 Σx^2 贡献，不需要任何跨 tile 状态。

8.7 Up_Proj + Gate_Proj + GELU_MUL

$G = X \cdot W_G$, $U = X \cdot W_U$, $Z = \text{GELU_tanh}(G) \text{ elementwise_mul } U$

三者均沿 N 维切分，core i 持有 W_G 与 W_U 各 512 列。

8.7.1 逐 tile 交替执行

为了让 GELU⊙ 能就地完成而不需要缓存整个 [51,4096]，G 与 U 按 tile 交替执行：

```
for n in 0..31:           // 每核 512 列 = 32 个 tile
  GEMM G(n) -> FACC_A      // 64 * 51 = 3,264 拍
  GEMM U(n) -> FACC_B      // 3,264 拍
  VOP_GELU_MUL_Q(FACC_A, FACC_B) -> ZBUF[n] + E_z[n] // 与下一对重叠
拍数 = 2 * 64 * 32 * 51 = 208,896
```

缓冲	规格	理由
U/G tile buffer	FP32 512b × 51 × 2 = 6.38 KiB	GELU⊙ 的两个输入必须同时在手
ZBUF + E_z	27.09 KiB	保存整个 [51,512] 的量化结果，供 down_proj 消费

HBM 布局要求：W_G 与 W_U 的同一个 n-tile 必须在 HBM 上相邻（§ 5.8.1 的 SEG_GU 交织），否则 WFIFO 会在两个远地址间跳转，破坏顺序突发。

8.7.2 GELU⊙ 的量化

Z 的块方向沿通道（down_proj 的归约维），因此每 16 个连续通道一个块，每核每行 32 个块。量化在 VOP_GELU_MUL_Q 的第二遍完成（第一遍算 GELU 与乘法并求块最大值）。

8.8 Down_Proj + 残差归约 + RMSNorm

```
core i 持 W_D[512i : 512i+512, 0:1024]
Kt = 512/16 = 32 , Nt = 1024/16 = 64
拍数 = 32 * 64 * 51 = 104,448
```

结构与 § 8.6 完全相同：K 切分 → FACC → CNET REDUCE → PRU（残差加 + Σx^2 + 块最大值）→ rsqrt → RMS Pass B。差别只在 Kt 从 16 变为 32，以及产出的 XHAT 服务于**下一层**的 input norm（而 § 8.6 的服务于本层的 post-attention norm）。

项	§ 8.6 (o_proj)	§ 8.8 (down_proj)
Kt / Nt	16 / 64	32 / 64
拍数	52,224	104,448
输入源	Q-Memory ($\hat{O}$)	ZBUF ($\hat{Z}$)
产出的 E_a 服务于	本层 post-attention norm	下一层 input norm

跨层的 XHAT 语义 层 ℓ 的 down_proj 之后产生的 XHAT 就是层 $\ell+1$ 的 RMS_a 输出。因此层边界上不需要额外的归一化事件，18 层共 36 次层内归一化 + 1 次最终归一化 = 37 次（§ 4.12）。

8.9 Final RMSNorm 与 action_out_proj（裁决 D8）

层 17 的 down_proj 与残差更新完成后，GCore 完成平方和、计算最终的 inv_sigma，并执行**同样的 3,264 拍归一化物化**。

```
y_t = final_rmsnorm(h^18[1:50])    # 只对 50 个 action token
v_t = action_out_proj(y_t)          # [50,1024] x [1024,32]
```

- final_norm 的 γ 折进 action_out_proj 的权重。
- **最终归一化不可省略，其结果不得用未归一化的残差替代。** 这是 V1.0 遗漏而 V1.2 补入的项（裁决 D8）。
- state token ($m = 0$) 不参与输出投影，但其残差行仍参与最终归一化的平方和？——**否**：
final_rmsnorm 是逐行的，state 行的归一化结果被丢弃。RTL 可选择跳过 $m = 0$ 行以省 64 拍，但基线为保持地址模式规整仍计算全部 51 行，只在 action_out_proj 处跳过。

8.10 步边界执行

0. 载入或保持块常量的 state 投影，并在 BF16 存储中初始化 x_t / 噪声
1. 对 50 个 action token 执行 action_in_proj
2. time embedding 由 LUT[k] 直接读出（10 个固定时刻，20 KB 表，裁决 D14）
3. 执行 action_time_mlp_in $\rightarrow$ GELU $\rightarrow$ action_time_mlp_out
4. 逻辑上拼接 action 与 time 特征，得到 $h_t^{0}[51,1024]$
5. 执行全部 18 层（含每层两次 RMS Pass B）
6. 执行 final_norm 与 action_out_proj
7. $x_{next} = x_t + dt * v_t$ ，在 FP32 中计算，只在写回 BF16 时舍入一次

8.10.1 动作块级流程

EHU : state_proj(s) $\rightarrow$ token 0；采样 $x_0 \sim N(0, I)[50, 32]$

// layer 0 的 $\sum(x^2)$ 与 E_a 在此一并算出

GCore: 预取 layer 0 的 prefix KV（433.5 KB）

for $k = 0..9$: // 10 个去噪步

$t_k = 1 - k/10$

EHU : time_emb $\leftarrow$ LUT[k] // 20 KB 离线表

执行步边界的 action_in_proj / time_mlp

for $L = 0..17$:

执行 § 9.7 的 MDESC 程序

GCore 后台预取 layer $L+1$ 的 KV

执行 final_norm 与 action_out_proj $\rightarrow v[50,32]$

```
EHU : x_{k+1} <- x_k + dt * v
HALT -> 中断，chunk[50,32] 就绪
```

8.10.2 time embedding LUT（裁决 D14）

项	值
时刻数	10（固定 $\tau_k = 1 - k/10$ ）
每时刻的 embedding	[1, 2048] BF16
表大小	$10 \times 2048 \times 2 \text{ B} = \mathbf{40 \text{ KB}}$ ；若只存半精度正弦基则 20 KB
运行时三角函数	零
版本绑定	表内容受兼容性清单的哈希绑定，启动前校验

若 PREFIX_RUNTIME_EN=1 或采用非基线的时刻表，则必须实例化 § 4.11 的 sin/cos SFU 通路并填充系数 ROM。基线不实例化。

8.11 γ 的离线折叠

8.11.1 为什么可以折叠

RMSNorm 的增益是逐通道的乘性标量，作用在归一化输出上：

$$X_hat[m,c] = n[m,c] * (1 + gamma[c])$$
$$Y[m,n] = \sum_c X_hat[m,c] * W[c,n]$$
$$= \sum_c n[m,c] * ((1 + gamma[c]) * W[c,n])$$
$$= \sum_c n[m,c] * W'[c,n] \quad \text{with } W'[c,n] = (1+gamma[c]) * W[c,n]$$

因此 γ 可以在离线打包时吸收进后继权重矩阵的**行**（输入通道方向）。

8.11.2 折叠表

归一化	γ 折进
层 ℓ 的 input norm	$W_Q[\ell]$ 、 $W_K[\ell]$ 、 $W_V[\ell]$ 的行
层 ℓ 的 post-attention norm	$W_gate[\ell]$ 、 $W_up[\ell]$ 的行
final norm	action_out_proj 的行

折叠后 W' 再执行 MXINT8-B16 打包（§ 4.4），因此 γ 不引入任何运行时开销，也不改变Scale的计算方式（PRU 的 P4b 直接取 $|h'|$ 的最大值）。

8.12 RoPE 映射与位置

$$x'[j] = x[j] * \cos(p * \theta_j) - x[j+128] * \sin(p * \theta_j)$$
$$x'[j+128] = x[j+128] * \cos(p * \theta_j) + x[j] * \sin(p * \theta_j)$$
$$\theta_j = \text{base}^{(-2j/d_h)} = 10000^{(-j/128)}, j = 0..127$$

项	值	说明
p（位置）	ROPE_POS_BASE + m，基线 816 + m	只有 51 个固定值（基线）
查找表规模	$51 \times 128 \times 2 \times 2 \text{ B} = \mathbf{25.5 \text{ KB}}$	cos 与 sin 各一份，BF16
寻址	$\text{addr} = m * 128 + j$	816 的偏移已在离线生成时吃进去
运行时三角函数	零（基线）	完全查表
需求速率	0.25 元素/拍	VPU 16 lane 有 64× 余量
乘加精度	FP32 （D5）	表系数为 BF16，乘积与累加为 FP32

旋转是保范变换，不改变 Q/K 的整体量级，因此量化Scale可以沿用旋转前的统计。VOP 需要两遍（x[j] 与 x[j+128] 交叉引用），必须先把两者读进寄存器再算，不能边读边写回原地址。
prefix 的 816 个 token 的 K 在 VLM 侧 prefill 时已经旋转过，存进 HBM 的就是旋转后的值，本加速器不再处理。

9 控制架构

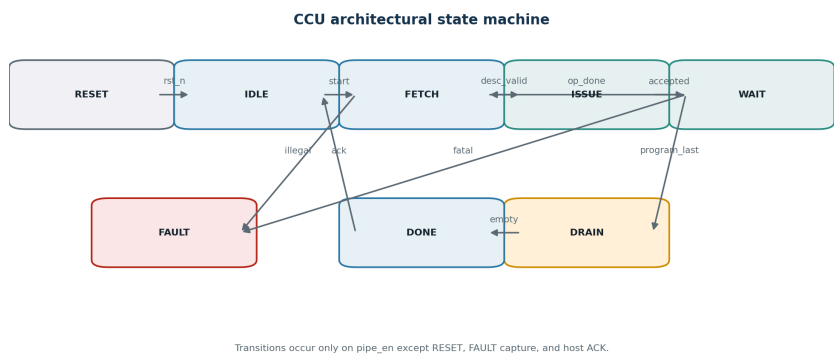


图 9-1 CCU 状态转移、正常退休与故障捕获

9.1 控制层次

控制分四层嵌套，全部为静态调度，无数据相关分支：

- chunk（动作块）
 - └─ step（去噪步）x 10
 - └─ 步边界程序（EHU + 4 个小 GEMM）

- └ layer x 18
- └ MDESC x 17（每条展开为最多 2,048 个 tile 操作）
- └ tile x (Kt x Nt)
- └ token x 51

CCU 持有前两层与第三层的计数器，TSQ 持有后两层。整个控制状态量为：

$$\text{CCU: } k[3:0] + L[4:0] + md[4:0] + blk[5:0] + state[3:0] = 24 \text{ bit}$$

blk 从 V1.0 的 4 bit 扩为 **6 bit**（0–54），这是裁决 D11 的直接后果。

9.2 CCU 状态机

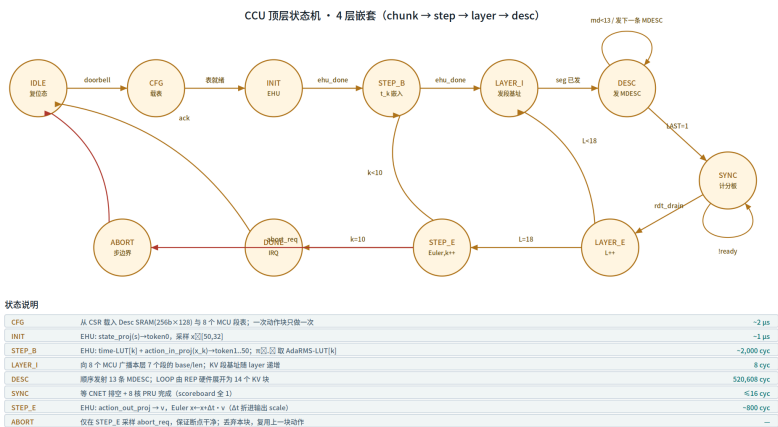


图 9-2 CCU 顶层状态机（源：V1.0 图 21，状态集已与 V1.2 合并）

编码	状态	Moore 输出 / 动作	转出条件	失败转移	典型驻留
4' h 0	IDLE	全部使能拉低，等待命令	start	—	—
4' h 1	CFG	载入 DESC SRAM + 8 × SEG TAB + GCore 段表 + prefix 掩码/计数 + CSR	cfg_done	非法参数 → ABORT	约 2 μs
4' h 2	BLOCK_INIT	ehu_call = INIT（state_proj、采样 x ₀ ）、KV 窗口校验	init_done	KV/ECC 故障 → ABORT	约 1 μs
4' h 3	STEP_EDGE	发射 4 条步边界 GEMM 与 time LUT 读取	step_edge_done	超时 → ABORT	80,800 拍
4' h 4	LAYER_I	向 8 个 MCU 与 GCore 广播段基址；启动 KV 预取	seg_ack 全 1	—	8 拍
4' h 5	DESC	顺序发射 MDESC[md]；LOOP 由 REP 硬件展开	LAST == 1	粘滞致命 → ABORT	521,727 拍

4' h 6	SYNC	等待 scoreboard[SYNC_ID] 全 1	sb_match	超时 → ABORT	≤ 16 拍
4' h 7	LAYER_E	KV-Memory 窗口乒乓；L++	组合	—	1 拍
4' h 8	FINAL	final_norm、action_out_proj、Euler 更新；采样 abort	step_done	超时 → ABORT	约 4,500 拍
4' h 9	DONE	发布输出与遥测，置中断	host_ack	—	—
4' h A	ABORT	冻结计算，作废部分结果，保留诊断，软复位所有核与 MCU/GCore	soft_reset	—	约 64 拍

9.3 SYNC 计分板

事件	生产者	消费者 / 清除规则
EV_XHAT_READY	GCore RMS/Quant	投影发射；最后一个消费者组退休时清除
EV_KV_SUFFIX_READY	CNET gather + KV 控制器	匹配 layer/step 的 Attention KVB 发起
EV_Q_READY	全部 8 个 PCore 的 Q 写者	QK 发起； 全核 AND 屏障
EV_EXP_B	全部 8 个 PCore 的 SFU	PV(B) 与 OACC_SCALE(B)
EV_O_READY	全部 8 个 A_FIN 引擎	输出投影发起
EV_CNET_DONE	CNET root / GCore sink	残差发布与下一次 RMS pass
EV_WFIFO_TILE	每 PCore 的 MCU	TSQ 就绪； 电平事件，非粘滞完成
EV_PROGRAM_DONE	CCU 排空检测器	主机完成与中断

计分板为 16 位，SYNC_ID 选择需要等待的位集合。SYNC 状态的驻留时间在标称情况下为 0（事件早已置位），最坏 16 拍。

9.4 TSQ 状态机

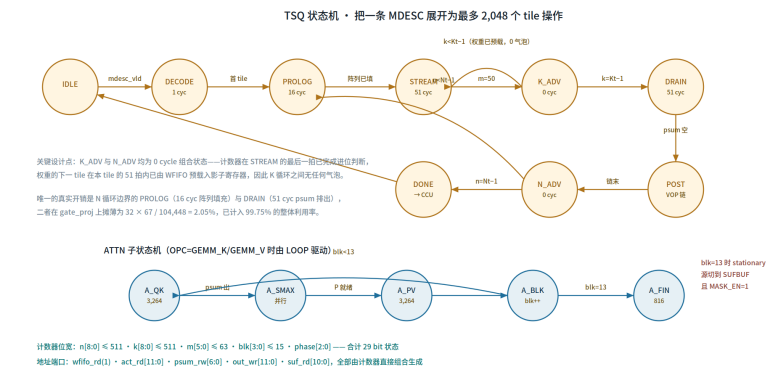


图 9-3 TSQ 主状态机与 Attention 子状态机（源：V1.0 图 22）

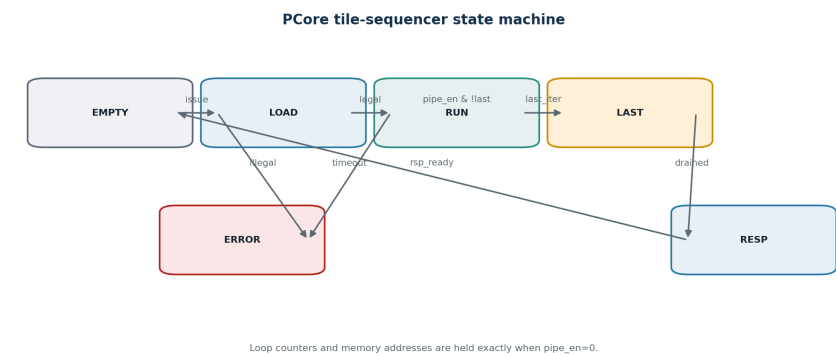


图 9-4 PCore TSQ 状态转移与受 pipe_en 门控的循环推进（源：V1.2 Fig.9-3）

状态	动作	转出条件
T_IDLE	等 MDESC FIFO 非空	desc_valid
T_DECODE	解包 256 bit, 做 § 9.8 的合法性检查, 快照到周期寄存器	合法 → T_PROLOG; 非法 → T_FAULT
T_PROLOG	载入首个 stationary tile (16 拍) + swap (1 拍)	17 拍后
T_RUN	双计数器推进: CNT_EX 发地址, CNT_WB 驱动写回	CNT_EX 到达 $Kt \cdot Nt \cdot M$
T_TAIL	等待最后一个写回标签退休 (≤ 11 拍)	tag_empty
T_RETIRE	置 desc_done, 更新计分板	组合
T_FAULT	置错误码, 停止发射	abort

Attention 的子状态机见 § 7.12。全部状态推进受 pipe_en_q 门控 (I9)。

9.5 MDESC 256 bit 格式

MDESC 为 256 bit, 由 CCU 广播, 每个 PCore 有深度 2 的接收 FIFO。

--	--	--	--

字段	位域	宽度	用途
OPCODE	255:250	6	见 § 9.6
FLAGS	249:240	10	LAST、转置、累加、RoPE、final-K、异常模式
SRC_A	239:232	8	激活/本地/广播端点 ID（附录 B）
SRC_B	231:224	8	权重/FIFO/KV 端点 ID
DST	223:216	8	本地/GCore/CNET 目的 ID
M_TILES	215:208	8	行数，以 16 元素 tile 计；行串行操作码除外
K_TILES	207:200	8	归约长度，以 tile 计
N_TILES	199:192	8	输出列，以 tile 计
BLK_ID	191:186	6	Attention 块 0–54
REP	185:180	6	重复次数减一；0 表示执行一次
LAYER	179:175	5	层 0–17（CCU 广播时覆写）
STEP	174:171	4	去噪步 0–9（CCU 广播时覆写）
BASE_A	170:139	32	源 A 的段相对字节地址
BASE_B	138:107	32	源 B 的段相对字节地址
BASE_D	106:75	32	目的的段相对字节地址
STRIDE_A	74:59	16	A 的行/tile 字节步长
STRIDE_B	58:43	16	B 的 tile 字节步长
STRIDE_D	42:27	16	目的的行/tile 字节步长
AUX	26:11	16	后处理、指数折叠、掩码模式、SFU 选择
QMODE	10:8	3	0 = MXINT8-B16；其余保留
CRC8	7:0	8	对 bits 255:8 的 CRC-8/ATM

LAYER / STEP / BLK_ID 在 CCU 广播时被覆写，因此 MDESC 表只需存 1 层 × 1 步的内容，DESC SRAM 只要 4 KB。

QMODE 的 MXINT4 与 BF16 矩阵模式在 V2.0 中**保留但不宣称支持**，直到数据通路、存储与验证产物齐备。

9.5.1 FLAGS 与 AUX 子域

父位域	子域	宽度	含义
FLAGS[9]	LAST	1	程序最后一条；退休后进入 DRAIN
FLAGS[8]	ACCUM	1	在操作码允许处读改写先前目的
FLAGS[7]	TRANS_B	1	使用受支持的转置权重 tile 走法
FLAGS[6]	ROPE	1	对 Q/K 目的施加 RoPE 后处理
FLAGS[5]	FINAL_K	1	本描述符终结一次预切分的 K 累加
FLAGS[4:3]	EXC_MODE	2	00 = 致命，01 = 饱和+粘滞，其余保留
FLAGS[2:0]	LOOP_ORDER	3	0 = MNK 基线；仅枚举的替代序
AUX[15:11]	POST_OP	5	NONE、CNET、MX_QUANT、GELU_MUL、A_FIN、RESID、OACC_SCALE、ROWMAX、EXP_Q
AUX[10:5]	EXP_FOLD	6（有符号）	附加 2 的幂Scale，如 QK 用 -4
AUX[4:3]	MASK_MODE	2	0 = 无，1 = prefix+suffix 基线，其余保留
AUX[2:0]	SFU_SEL	3	EXP、RECIP、RSQRT、GELU、SIN、COS、none

9.6 操作码表

操作码	值	必需端点	语义 / 完成条件
OP_GEMM	0x00	SRC_A, SRC_B, DST	分块 MX GEMM；带标签的最终写回后完成
OP_QK	0x01	Q, KVB, SBUF	BLK_ID 的注意力分数 GEMM
OP_PV	0x02	PBUF, KVB, OACC	BLK_ID 的概率-值 GEMM
OP_VPU	0x08	本地/GCore 向量口	由 POST_OP / SFU_SEL 选择的操作
OP_CNET_REDUCE	0x10	8 PCore → GCore	固定树 FP32 归约
OP_CNET_GATHER	0x11	8 PCore → GCore	字节精确的不相交分片聚集
OP_XBC	0x18	GCore → 8 PCore	锁步激活广播

OP_KVB	0x19	GCore → 8 PCore	锁步 K/V 块广播
OP_KVPREF	0x1A	GCore ← HBM pch8	非阻塞预取下一层 prefix KV 到备用窗口
OP_SYNC	0x20	计分板事件掩码	所选事件全部退休时完成
OP_SEGMENT	0x28	段表	范围检查后设置活动基址 ID
OP_EHU	0x30	GCore EHU	调用 EHU 子程序（入口由 AUX 指定）；约 600 拍
OP_DBGSNAP	0x38	全部	缓冲区快照写回 pch8；可变拍数，仅调试
OP_NOP	0x3D	无	对齐 / 占位，1 拍
OP_HALT	0x3E	CCU	动作块结束，置中断
OP_END	0x3F	无	程序末条；CRC/flags 必须合法

9.7 单层 MDESC 程序

代码块

```
1 // LAYER / STEP / BLK_ID 由 CCU 广播时覆写；表中只存一份
2 // # OPCODE          SRC_B  SRC_A  DST      Kt  Nt  BLK  POST_OP
   拍数
3   -- OP_SEGMENT      ---      ---      ---      --  --  ---  ---
   8
4   0 OP_VPU            ---      RESID  XHAT      --  --  ---  RMS_PASS_B
   3,264 // input norm
5   1 OP_GEMM           WFIFO    XBC      local    64   4  ---  ROPE_K|MX_QUANT
   13,056 // kv_proj
6   2 OP_CNET_GATHER    ---      local    KVMEM     --  --  ---  ---
   3,264 // -> GCore
7   3 OP_GEMM           WFIFO    XBC      Q_MEM     64  16  ---  ROPE_Q|MX_QUANT
   52,224 // q_proj
8   4 OP_VPU            ---      ---      ---      --  --  ---  SMAX_INIT
   51
9   5 OP_QK             KVB      Q_MEM     SBUF0     16   1   +0  ROWMAX
   816 // QK(0)
10  6 OP_QK             KVB      Q_MEM     SBUF1     16   1   +1  ROWMAX
   816 // QK(1) || EXP(0)
11  7 OP_PV             KVB      PBUF0     OACC       1  16   +0  ---
   816 // PV(0) || EXP(1)
12  8 REP = 53, blk 自增，展开 #9 - #10
13  9 OP_QK             KVB      Q_MEM     SBUF       16   1   +1  ROWMAX
   816 // QK(b+1) || SCALE(b)
```

14	10	OP_PV	KVB	PBUF	OACC	1	16	+0	---
	816	// PV(b)		EXP(b+1)					
15	11	OP_VPU	---	OACC	OACC	--	--	+54	OACC_SCALE
	816	// ★ alpha54 暴露尾							
16	12	OP_PV	KVB	PBUF	OACC	1	16	+54	---
	816	// PV(54)		A_FIN					
17	13	OP_VPU	---	OACC	Q_MEM	--	--	---	A_FIN
	(重叠)								
18	14	OP_GEMM	WFIFO	Q_MEM	CNET	16	64	---	CNET RESID
	52,224	// o_proj							
19	15	OP_VPU	---	RESID	XHAT	--	--	---	RMS_PASS_B
	3,264	// post-attn norm							
20	16	OP_GEMM	WFIFO	XBC	ZBUF	64	32	---	GELU_MUL MX_QUANT
	208,896	// gate+up 交织							
21	17	OP_GEMM	WFIFO	ZBUF	CNET	32	64	---	CNET RESID LAST
	104,448	// down_proj							
22	18	OP_SYNC	SYNC_ID = 1						
	~40								

9.8 描述符合法性矩阵

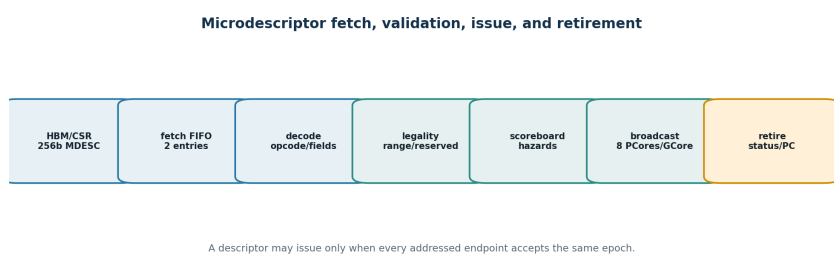


图 9-5 描述符取指、合法性、计分板、广播与退休流程

检查	合法条件	错误码
CRC	CRC8(bits 255:8) 等于 CRC8 字段	ERR_DESC_CRC
QMODE	QMODE == 0	ERR_QMODE
维度	GEMM 族的 M_TILES/K_TILES/N_TILES 非零	ERR_DIM_ZERO
Attention 块	QK/PV/EXP/SCALE 的 BLK_ID < 55；其余（除非忽略）BLK_ID == 0	ERR_BLK_RANGE
层/步	运行时程序满足 LAYER < 18 且 STEP < 10	ERR_CONTEXT_RANGE
端点	SRC/DST 编码对所选操作码与 PCore 角色被允许	ERR_ENDPOINT

地址范围	base + 最大索引偏移 落在所选段限内且不发生 32 位回绕	ERR_ADDRESS
Flags/AUX	保留组合为零；POST_OP 对该操作码被允许	ERR_RESERVED
循环边界	派生的最终地址落在本地 SRAM 与 WFIFO tile 计数内	ERR_LOCAL_RANGE
掩码一致性	MASK_MODE == 1 当且仅当 BLK_ID >= 51 (PREFIX_RUNTIME_EN=0 时)	ERR_MASK_MODE
OACC 互斥	OP_PV 与 POST_OP=OACC_SCALE 不得在同一周期占用 OACC	ERR_OACC_CONFLICT

9.9 停顿、超时与恢复

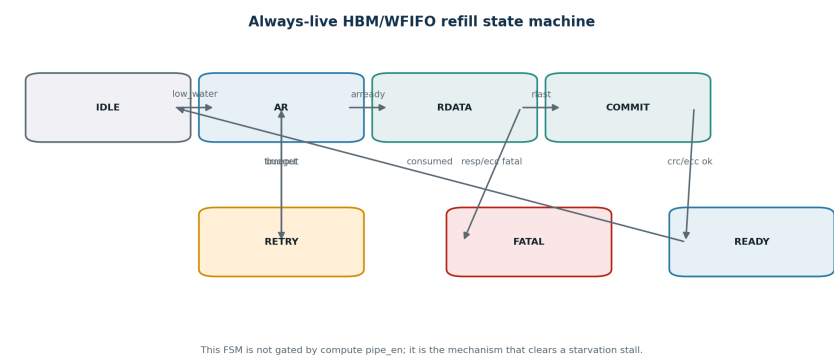


图 9-6 always-live 的 HBM/WFIFO 补给与重试状态机（源：V1.2 Fig.9-4）

事件	检测	动作	遥测
WFIFO 低水位	wfifo_level < 完整 tile 阈值	拉低 pcore_ready → 全局冻结	CYC_STALL_WEIGHT、FIFO_MIN[i]
KVB 未就绪	!kvb_ready	同上	CYC_STALL_KV
CNET 反压	!cnet_ready	冻结生产者与树	CYC_STALL_CNET
AXI 无响应	每通道重试计时器超阈	置 ERR_AXI_TIMEOUT → ABORT	通道 ID + 地址快照
描述符超时	DESC 状态驻留 > CSR_DESC_TIMEOUT	ABORT	md、LAYER、STEP 快照
ECC 不可纠	SRAM wrapper 报 ecc_uncorr	粘滞致命 → ABORT	地址与存储器 ID
格式故障	编码器遇 NaN/Inf 且 EXC_MODE=00	粘滞 FORMAT_FATAL → ABORT	张量 ID、行、块

9.10 进展与死锁证明义务（规范性）

实现必须在存储服务契约下满足下列有限进展论证：

1. 空的 WFIFO 保持其 MCU 补给请求为活动；
2. 被接受的 AXI 响应在**不依赖 pipe_en** 的情况下推进 FIFO 写指针；
3. 达到完整 tile 阈值后重新置起 pcore_ready；
4. 寄存后的就绪树抬起 pipe_en；
5. 一个计算周期消费数据或朝退休推进。

CNET 反压同理可解，因为其目的端就绪通路是寄存的，不组合依赖当前源的 valid。形式化的活性假设可以约束 AXI 为”最终响应”，但**安全性性质不得假设 pipe_en 恒高**。

9.11 CCU 与 PCore 的发射/响应契约

见 § 12.4 的端口定义。要点：

- CCU 广播 MDESC 到 8 个 PCore 的深度 2 FIFO；8 核**必须在同一周期接受**同一条描述符。
- PCore 的 desc_done 是 8 路 AND 后进入计分板，任何一核落后都会阻塞 SYNC。
- epoch 标签（4 bit）随描述符下发，用于在 abort/reset 后拒绝过期响应。
-

10 模块接口规范

本章的端口列表是规范性的。位宽由附录 A 的参数包派生，禁止硬编码。

12.1 顶层时钟、复位、命令与中断

信号	方向/宽度	时钟/复位	含义
clk_core	I / 1	—	计算域与 always-live 核心域时钟
rst_core_n	I / 1	clk_core	低有效，异步置位、同步撤销
clk_hbm	I / 1	—	HBM AXI 时钟
rst_hbm_n	I / 1	clk_hbm	HBM 域复位
clk_cfg	I / 1	—	CSR/控制时钟
rst_cfg_n	I / 1	clk_cfg	CSR 域复位
cmd_valid	I / 1	clk_cfg	命令负载有效

cmd_ready	O / 1	clk_cfg	命令队列可接受
cmd_data	I / 256	clk_cfg	§ 13.1 定义的作业命令记录
rsp_valid	O / 1	clk_cfg	完成/中止记录有效
rsp_ready	I / 1	clk_cfg	主机接受响应
rsp_data	O / 128	clk_cfg	作业 ID、状态、周期数、征状摘要
irq_done	O / 1	clk_cfg	电平中断；由 W1C 状态清除
irq_error	O / 1	clk_cfg	致命完成的电平中断

12.2 HBM AXI4 只读主端口

信号组	方向/宽度	握手 / 要求
m_axi_arid	O / AXI_ID_W	在 arvalid && !arready 期间与 araddr/arlen 一同保持稳定
m_axi_araddr	O / 64	字节地址；自然对齐到 32 字节
m_axi_arlen	O / 8	突发 beat 数减一；不跨 4 KiB
m_axi_arsize	O / 3	常量 5（32 字节 beat）
m_axi_arburst	O / 2	常量 INCR（2’ b01）
m_axi_arvalid / arready	O / I，各 1	两者同为 1 时地址被接受
m_axi_rid	I / AXI_ID_W	匹配一个未完成请求
m_axi_rdata	I / 256	小端数据 beat
m_axi_rresp	I / 2	仅接受 OKAY；SLVERR/DECERR 为致命
m_axi_rlast	I / 1	必须与跟踪的最终 beat 一致
m_axi_rvalid / rready	I / O，各 1	两者同为 1 时响应被接受；CDC FIFO 有空间时 rready 保持置起

该组实例化 **9 次**。主 0–7 永久绑定 PCore 0–7 的权重流；主 8 服务共享 KV 与 EHU 传输。**在带某 ID 的先前请求收到其 rlast beat 之前，禁止复用该 ID。** ID 或 beat 计数意外的响应锁存 ERR_AXI_PROTOCOL。

12.3 标准 ready/valid 通道

规则	规范性行为
传输	负载仅在上升沿的 valid && ready 时传输
源稳定性	置起 valid 后，源保持 valid 与负载直到传输完成
目的独立性	目的可以先于 valid 置起 ready，且 不得组合依赖负载数据
无丢失/重复	每次源传输恰有一次目的接受
全局计算通道	XBC/KVB/CNET/本地发射的传输条件加上 && pipe_en； pipe_en=0 时 valid 与负载保持
Always-live 通道	HBM 响应与 FIFO 补给 不使用 pipe_en
复位	valid 清零； ready 在初始化完成前可为 0

```
accept = valid && ready;           // always-live 通道
commit = valid && ready && pipe_en;  // 计算通道
```

```
if (valid && !ready) begin
    valid <= 1'b1;
    payload <= $stable(payload);
end
if (!pipe_en) begin
    compute_valid <= compute_valid;
    compute_state <= compute_state;
end
```

12.4 CCU ↔ PCore 发射/响应接口

信号	方向/宽度	语义
issue_valid	I / 1	描述符上下文对本 PCore 有效
issue_ready	O / 1	TSQ 空且全部启动资源可用
issue_desc	I / 256	已校验的 MDESC；在被接受前保持稳定
issue_epoch	I / 4	用于 abort/过期响应保护的世代标签
issue_core_mask	I / 8	本核仅在自身掩码位置起时接受
rsp_valid	O / 1	操作完成或本地故障
rsp_ready	I / 1	CCU 响应收集器接受

rsp_epoch	O / 4	从被接受的发射复制
rsp_status	O / 8	DONE 加本地错误分类
rsp_count	O / 24	本操作消耗的使能计算周期数
pcore_ready	O / 1	当前操作可推进一个计算周期
pcore_fatal	O / 1	向 always-live pipe 控制器发出的粘滞致命请求

12.5 XBC 激活广播接口

信号	方向/宽度	语义
xbc_valid	GCore O / 1	一个激活块可用
xbc_ready[7:0]	PCore O / 8	逐目的的接收能力；源传输取所选核的 AND
xbc_q	GCore O / 256	两个相邻的 16 码块，或一个 32 字节打包激活 beat
xbc_e	GCore O / 16	两个 E8M0 Scale字节；只有一个块时未用位为零
xbc_row	GCore O / 6	suffix 行 0–50
xbc_blk	GCore O / 6	D 块 0–63
xbc_lane_mask	GCore O / 32	有效码字节；基线全 1
xbc_last	GCore O / 1	本次广播世代的最后一块
xbc_epoch	GCore O / 4	描述符世代标签

12.6 KVB 的 K/V 广播接口

信号	方向/宽度	语义
kvb_valid	GCore O / 1	一个 K 或 V 的 16 码特征块有效
kvb_ready[7:0]	PCore O / 8	所选 PCore 必须一同接受
kvb_q	GCore O / 128	16 个有符号 INT8 码
kvb_e	GCore O / 8	E8M0 ScaleScale
kvb_kind	GCore O / 1	0 = K, 1 = V
kvb_blk_id	GCore O / 6	物理 key 块 0–54

kvb_key_lane	GCore O / 4	块内 key 0-15
kvb_feat_blk	GCore O / 4	特征块 0-15
kvb_valid_mask	GCore O / 16	当前块内物理 key 的有效性（PREFIX_RUNTIME_EN=1 时使用）
kvb_last	GCore O / 1	本描述符的最后一个特征/key 条目
kvb_epoch	GCore O / 4	描述符世代标签

12.7 CNET leaf/root 接口（512b）

信号	方向/宽度	语义
leaf_valid[7:0]	PCore O / 8	每个所选核呈现一个 16 lane 的 FP32 向量
leaf_ready[7:0]	CNET O / 8	所选叶子在 同一计算周期 内传输
leaf_data[7:0]	PCore O / 8 × 512	每核 16 个 IEEE FP32 lane
leaf_tag[7:0]	PCore O / tag	除 PCore ID 外标签必须一致
root_valid	CNET O / 1	归约或聚集输出有效
root_ready	GCore I / 1	目的可接受输出
root_data	CNET O / 512	归约后的向量或当前聚集分片
root_tag	CNET O / tag	匹配的操作上下文
cnet_mode	CCU I / 2	0 = reduce, 1 = gather, 其余保留
cnet_ready	CNET O / 1	流水可推进一个全局周期

12.8 SRAM wrapper 接口

信号	方向/宽度	语义
clk	I / 1	本地同步时钟
ce	I / 1	宏时钟使能；对计算存储器绑到功能请求 && pipe_en
rd_en	I / 1	ce=1 时发起读
rd_addr	I / ADDR_W	寄存后的地址；边界须断言

rd_data	O / DATA_W	RD_LATENCY 个使能周期后有效
rd_valid	O / 1	与 rd_data 对齐；全局停顿期间由 wrapper 保持
wr_en	I / 1	ce=1 时提交写
wr_addr	I / ADDR_W	写地址
wr_data	I / DATA_W	写负载
wr_mask	I / DATA_W/8	字节写掩码
ecc_corr / ecc_uncorr	O / 各 1	在被接受的已纠正/不可纠正读上产生脉冲

11 RTL 实现模板

模板状态 下列 SystemVerilog 片段定义命名、参数、状态边界与断言挂钩，作为实现起点。浮点算子、SRAM 宏、CDC FIFO 与 AXI 引擎必须替换为经过认证的项目 IP，同时保持这些契约。

15.1 核心参数与类型包

代码块

```
1 package dea8_pkg;
2   // ---- 改这五个即可切换设计点 ----
3   parameter int unsigned N_CORE    = 8;
4   parameter int unsigned TILE      = 16;    // = TILE_R = TILE_C = MX 块大小 =
      B_kv
5   parameter int unsigned M_MAX     = 64;    // 流式维上限 (pi0 实际 51)
6   parameter int      MXU_KIND = 0;    // 0 = adder-tree, 1 = systolic
7
8   // ---- 模型形状 ----
9   parameter int unsigned D_MODEL   = 1024;
10  parameter int unsigned D_HEAD    = 256;
11  parameter int unsigned D_FFN     = 4096;
12  parameter int unsigned N_LAYER   = 18;
13  parameter int unsigned PREFIX_CAP = 816;
14  parameter int unsigned SUFFIX_LEN = 51;
15  parameter int unsigned N_STEP    = 10;
16  parameter int unsigned ACT_HORIZON= 50;
17  parameter int unsigned ACT_DIM   = 32;
18
19  // ---- 推导量 (不要手改) ----
```

```

20     parameter int unsigned LOGICAL_SEQ = PREFIX_CAP + SUFFIX_LEN;
    // 867
21     parameter int unsigned PHYS_SEQ    = ((LOGICAL_SEQ+TILE-1)/TILE)*TILE;
    // 880
22     parameter int unsigned N_KV_BLOCK  = PHYS_SEQ/TILE;
    // 55
23     parameter int unsigned BLK_W      = $clog2(N_KV_BLOCK);
    // 6
24     parameter int unsigned SUFFIX_BLK0 = PREFIX_CAP/TILE;
    // 51
25
26     parameter int unsigned MXU_LAT     = (MXU_KIND==0) ? 6 : (2*TILE);
27     parameter int unsigned DEQACC_LAT = 5;
28     parameter int unsigned PIPE_DRAIN  = MXU_LAT + DEQACC_LAT;
    // 11
29
30     parameter int          MX_E_BIAS    = 127;
31     parameter int          MX_FRAC_BITS = 6;
32     parameter int          DOT_EXP_OFFSET = 2*MX_E_BIAS + 2*MX_FRAC_BITS;
    // 266
33     parameter int          QK_EXP_FOLD  = -4;
    // 1/sqrt(256)
34
35     // ---- 位宽 (全部由 TILE 派生) ----
36     parameter int unsigned DW_ST      = TILE*8;    // 128 stationary
37     parameter int unsigned DW_ACT     = TILE*8;    // 128 activation
38     parameter int unsigned DW_PSUM    = TILE*32;   // 512 部分和
39     parameter int unsigned DW_VEC     = TILE*32;   // 512 VPU
40     parameter int unsigned DW_RES     = TILE*16;   // 256 残差 (BF16)
41     parameter int unsigned DW_SCALE   = TILE*8;    // 128 = 16 x E8M0
42     parameter int unsigned DW_XBC     = TILE*16;   // 256 X 广播
43     parameter int unsigned DW_CNET    = TILE*32;   // 512 CNET (裁决 D9)
44     parameter int unsigned N_LANE     = TILE;      // 16 VPU / DEQACC lane
45     parameter int unsigned N_SFU      = 2;
46     parameter int unsigned N_PRU      = 4;
47
48     // ---- 存储深度 ----
49     parameter int unsigned M_WORDS     = SUFFIX_LEN*(D_MODEL/TILE); // 3264
50     parameter int unsigned OACC_WORDS  = SUFFIX_LEN*(D_HEAD/TILE);  // 816
51     parameter int unsigned ZBUF_WORDS  = SUFFIX_LEN*(D_FFN/(TILE*N_CORE)); //
1632
52     parameter int unsigned RESID_BYTES = SUFFIX_LEN*D_MODEL*2;      // 104448
53     parameter int unsigned XHAT_BYTES  = SUFFIX_LEN*D_MODEL;        // 52224
54     parameter int unsigned WEIGHT_TILE_B = TILE*TILE + TILE;        // 272
55     parameter int unsigned AXI_BEAT_B   = 32;
56     parameter int unsigned TILE_AXI_BEATS = (WEIGHT_TILE_B+AXI_BEAT_B-
1)/AXI_BEAT_B; // 9

```

```

57     parameter int unsigned FIFO_ENTRY_B    = 16;
58     parameter int unsigned TILE_FIFO_ENT    = WEIGHT_TILE_B/FIFO_ENTRY_B;
        // 17
59     parameter int unsigned WFIFO_DEPTH      = 512;
60     parameter int unsigned WFIFO_MIN_DEPTH = 208;
61     parameter int unsigned KVFIFO_DEPTH     = 128;
62     parameter int unsigned XFIFO_DEPTH      = 128;
63
64     typedef logic [31:0] fp32_t;
65     typedef logic [15:0] bf16_t;
66     typedef struct packed { logic [7:0] e; logic signed [TILE-1:0][7:0] q; }
mxblk_t;
67
68     typedef enum logic [5:0] {
69         OP_GEMM          = 6'h00, OP_QK          = 6'h01, OP_PV          = 6'h02,
70         OP_VPU           = 6'h08,
71         OP_CNET_REDUCE   = 6'h10, OP_CNET_GATHER = 6'h11,
72         OP_XBC           = 6'h18, OP_KVB         = 6'h19, OP_KVPREF    = 6'h1A,
73         OP_SYNC          = 6'h20, OP_SEGMENT    = 6'h28,
74         OP_EHU           = 6'h30, OP_DBGSNAP     = 6'h38,
75         OP_NOP           = 6'h3D, OP_HALT        = 6'h3E, OP_END       = 6'h3F
76     } opcode_e;
77
78     typedef struct packed {
79         logic [3:0]      epoch;
80         opcode_e         op;
81         logic [7:0]      dst;
82         logic [5:0]      row;
83         logic [7:0]      nt, kt;
84         logic [2:0]      head;
85         logic [BLK_W-1:0] blk;
86         logic [TILE-1:0] lane_mask;
87         logic [4:0]      post_op;
88         logic signed [5:0] exp_fold;
89         logic            final_k, last;
90     } pipe_tag_t;
91
92     // DEQACC 内部短标签 (裁决 D4 后无 alpha_en, 13 bit)
93     typedef struct packed {
94         logic [1:0]  acc_sel;    // FACC_A / FACC_B / OACC / SBUF
95         logic [9:0]  acc_addr;
96         logic        tile_last;
97     } deq_tag_t;
98
99     initial begin
100         assert (TILE == 16 && PHYS_SEQ == 880 && N_KV_BLOCK == 55);
101         assert (DOT_EXP_OFFSET == 266 && BLK_W == 6);

```

```

102     assert (SUFFIX_BLK0 == 51);
103     assert (PIPE_DRAIN == 11);
104     assert (ZBUF_WORDS == 1632);
105     end
106 endpackage

```

把 TILE 改为 32，则所有位宽同步翻倍、MXINT8 块大小同步变 32、其余 RTL 一行不用动，即得 § 11.9 的升级设计点。**块大小必须等于 TILE**，否则同构性破坏、PSUM 寄存器堆需要加回。

15.2 ready/valid 接口

代码块

```

1  interface dea8_rv_if #(parameter int W = 128) (input logic clk, rst_n);
2      logic valid, ready;
3      logic [W-1:0] payload;
4      modport source (output valid, payload, input ready);
5      modport sink (input valid, payload, output ready);
6      `ifndef SYNTHESIS
7          ap_stable: assert property (@(posedge clk) disable iff(!rst_n)
8              valid && !ready | => valid && $stable(payload));
9      `endif
10 endinterface

```

15.3 全局 pipe 控制器

代码块

```

1  module dea8_pipe_ctrl #(parameter int N = 8) (
2      input logic clk_core, rst_core_n, run_active, abort_async,
3      input logic [N-1:0] pcore_ready,
4      input logic gcore_ready, cnet_ready, xbc_ready, kvb_ready,
5      output logic pipe_en,
6      output logic abort_pending);
7
8      logic all_ready_d;
9      always_comb all_ready_d = &pcore_ready & gcore_ready & cnet_ready
10         & xbc_ready & kvb_ready;
11
12      always_ff @(posedge clk_core or negedge rst_core_n) begin
13          if (!rst_core_n) begin
14              pipe_en <= 1'b0; abort_pending <= 1'b0;
15          end else begin
16              abort_pending <= abort_pending | abort_async;

```

```

17     pipe_en      <= run_active & all_ready_d & !abort_pending &
    !abort_async;
18     if (!run_active) abort_pending <= 1'b0;
19     end
20     end
21 endmodule

```

就绪扇入必须在物理上流水化，同时不改变**单周期原子性**规则。若需要多级寄存，源端必须声明足够的 skid 容量，使寄存后的决策不会溢出任何端点。上述简化模板代表架构边界，**不是**对未做时序的组合树的豁免。

15.4 计算状态保持模式

代码块

```

1  always_ff @(posedge clk_core or negedge rst_core_n) begin
2      if (!rst_core_n) begin
3          state_q <= ST_IDLE; m_q <= '0; kt_q <= '0; valid_q <= 1'b0;
4      end else if (fatal_commit) begin
5          state_q <= ST_FAULT; valid_q <= 1'b0;
6      end else if (pipe_en) begin
7          state_q <= state_d; m_q <= m_d; kt_q <= kt_d; valid_q <= valid_d;
8      end
9  end
10 assign sram_wr_en = pipe_en & wr_valid & !abort_pending;

```

15.5 MDESC 解包与合法性骨架

代码块

```

1  typedef struct packed {
2      opcode_e      opcode;  logic [9:0]  flags;
3      logic [7:0]   src_a, src_b, dst;
4      logic [7:0]   m_tiles, k_tiles, n_tiles;
5      logic [5:0]   blk_id, rep;
6      logic [4:0]   layer;    logic [3:0]  step;
7      logic [31:0]  base_a, base_b, base_d;
8      logic [15:0]  stride_a, stride_b, stride_d, aux;
9      logic [2:0]   qmode;    logic [7:0]  crc8;
10 } mdesc_t;
11
12 mdesc_t d; assign d = mdesc_t'(desc_bits);
13
14 always_comb begin

```

```

15   illegal = 1'b0;  err = ERR_NONE;
16   if (crc8_atm(desc_bits[255:8]) != d.crc8)          begin illegal=1;
err=ERR_DESC_CRC;      end
17   else if (d.qmode != 3'd0)          begin illegal=1;
err=ERR_QMODE;      end
18   else if (is_gemm(d.opcode) &&
19           !(|d.m_tiles && |d.k_tiles && |d.n_tiles))  begin illegal=1;
err=ERR_DIM_ZERO;    end
20   else if (is_attn(d.opcode) && d.blk_id >= N_KV_BLOCK)begin illegal=1;
err=ERR_BLK_RANGE;    end
21   else if (d.layer >= N_LAYER || d.step >= N_STEP)    begin illegal=1;
err=ERR_CONTEXT_RANGE; end
22   // 掩码一致性 (裁决 D6/D11)
23   else if (is_qk(d.opcode) &&
24           (d.aux[4:3]==2'd1) != (d.blk_id >= SUFFIX_BLK0) &&
!prefix_runtime_en)
25                                           begin illegal=1;
err=ERR_MASK_MODE;    end
26   end

```

15.6 TSQ 计数器骨架

代码块

```

1  always_comb begin
2    m_d = m_q;  nt_d = nt_q;  kt_d = kt_q;  op_done_d = 1'b0;
3    if (advance) begin
4      if (kt_q + 1 < k_tiles_q) kt_d = kt_q + 1;
5      else begin
6        kt_d = '0;
7        if (nt_q + 1 < n_tiles_q) nt_d = nt_q + 1;
8        else begin
9          nt_d = '0;
10         if (m_q + 1 < m_count_q) m_d = m_q + 1;
11         else op_done_d = 1'b1;
12       end
13     end
14   end
15 end
16 assign advance = pipe_en & operand_valid & result_capacity;
17 assign final_k = (kt_q == k_tiles_q - 1);

```

写回侧必须有**第二组同构计数器** CNT_WB，相位滞后 PIPE_DRAIN 拍（§ 6.3.1 约束 ①）。

15.7 MXU / DEQACC 边界

代码块

```
1  typedef struct packed {
2      pipe_tag_t                tag;
3      logic signed [TILE-1:0][31:0] psum;
4      logic [7:0]                ea, eb;
5  } mxu_rsp_t;
6
7  logic signed [9:0] scale_exp;
8  always_comb scale_exp = $signed({1'b0, mxu_rsp.ea})
9                      + $signed({1'b0, mxu_rsp.eb})
10                     - DOT_EXP_OFFSET           // 266, 裁决 D1
11                     + $signed(mxu_rsp.tag.exp_fold); // QK 为 -4
12
13 // 认证的 FP32 IP 实现:
14 //  partial_fp = int32_to_fp32(psum) * pow2(scale_exp)
15 //  facc_next  = fp32_add(facc_old, partial_fp) — 固定 K-tile 顺序
16 assign facc_we = pipe_en & rsp_valid & !rsp.tag.final_k;
17 assign wb_we   = pipe_en & rsp_valid &  rsp.tag.final_k;
18 // 注意: DEQACC 内不得有任何 alpha 乘法器 (裁决 D4)
```

15.8 MX 编码器

15.9 VPU OACC α 缩放骨架

代码块

```
1  typedef enum logic [1:0] {VS_IDLE, VS_READ, VS_WRITE, VS_DONE} vs_e;
2
3  always_ff @(posedge clk_core or negedge rst_core_n) begin
4      if (!rst_core_n) begin vs_q <= VS_IDLE; addr_q <= '0; end
5      else if (pipe_en) begin
6          unique case (vs_q)
7              VS_IDLE : if (start)                begin vs_q <= VS_READ; addr_q <= '0; end
8              VS_READ  : if (oacc_rd_valid)         vs_q <= VS_WRITE;
9              VS_WRITE : if (mul_rsp_valid) begin
10                  if (addr_q == OACC_WORDS-1) vs_q <= VS_DONE;
11                  else begin addr_q <= addr_q + 1'b1; vs_q <= VS_READ; end
12              end
13              VS_DONE : if (rsp_ready)              vs_q <= VS_IDLE;
14          endcase
15      end
16  end
17  assign oacc_wr_en = pipe_en & (vs_q == VS_WRITE) & mul_rsp_valid;
```

```

18 // 流水实现可重叠 READ/WRITE，同时保持 1 word/拍的吞吐。
19 // 乘法器必须真是 FP32（裁决 D5）。

```

15.10 CNET 固定树骨架 (512b)

代码块

```

1  for (genvar lane = 0; lane < TILE; lane++) begin : g_lane
2      fp32_add u10 (.a(leaf[0][lane]), .b(leaf[1][lane]), .y(l1[0][lane]));
3      fp32_add u11 (.a(leaf[2][lane]), .b(leaf[3][lane]), .y(l1[1][lane]));
4      fp32_add u12 (.a(leaf[4][lane]), .b(leaf[5][lane]), .y(l1[2][lane]));
5      fp32_add u13 (.a(leaf[6][lane]), .b(leaf[7][lane]), .y(l1[3][lane]));
6      fp32_add u20 (.a(l1[0][lane]), .b(l1[1][lane]), .y(l2[0][lane]));
7      fp32_add u21 (.a(l1[2][lane]), .b(l1[3][lane]), .y(l2[1][lane]));
8      fp32_add u30 (.a(l2[0][lane]), .b(l2[1][lane]), .y(root[lane]));
9  end
10 // 每级寄存; valid 与 tag 寄存器使用同一个 pipe_en 门控。
11 // 综合工具不得重排这些加法（结合次序是架构可见的，§4.8）。

```

15.11 SRAM wrapper 骨架

代码块

```

1  module dea8_sram_wrap #(parameter AW = 10, DW = 128, RD_LATENCY = 1) (...);
2      // 综合配置中用 foundry 宏替换 mem 数组。
3      logic [DW-1:0] mem [0:(1<<AW)-1];
4      logic [DW-1:0] rd_pipe [0:RD_LATENCY-1];
5
6      always_ff @(posedge clk) begin
7          if (ce && wr_en) mem[wr_addr] <= merge_bytes(mem[wr_addr], wr_data,
8              wr_mask);
9          if (ce && rd_en) rd_pipe[0] <= mem[rd_addr];
10         if (ce) for (int i = 1; i < RD_LATENCY; i++) rd_pipe[i] <= rd_pipe[i-1];
11     end
12     assign rd_data = rd_pipe[RD_LATENCY-1];
13
14     ap_collision: assert property (@(posedge clk)
15         !(ce && rd_en && wr_en && rd_addr == wr_addr));
16 endmodule

```

15.12 Attention 掩码骨架

代码块

```

1 // blk >= SUFFIX_BLK0 时非平凡; jj 为 suffix 内全局列号
2 logic [5:0] jj;
3 logic      masked;
4
5 always_comb begin
6     jj      = ((blk_id - SUFFIX_BLK0) << 4) + j;
7     masked = prefix_runtime_en && (blk_id < SUFFIX_BLK0)
8             ? ~kvb_valid_mask[j]
9             : (blk_id >= SUFFIX_BLK0)
10              ? ( (jj >= SUFFIX_LEN) || ((m == 6'd0) && (jj >= 6'd1)) )
11              : 1'b0;
12     score_masked = masked ? NEG_INF_FP32 : score;
13 end
14
15 `ifndef SYNTHESIS
16 // state 行不得看到任何 action token (裁决 D6 的可观测判据)
17 ap_state_blind: assert property (@(posedge clk) disable iff(!rst_n)
18     (m == 0 && blk_id >= SUFFIX_BLK0 && jj >= 1) |-> (p_out == 32'h0000_0000));
19 `endif

```

12 验证与形式化闭环

验证使用四个互相校验的参考：**软件模型**定义功能张量；**位精确数值库**定义转换与 SFU 近似；**事件周期模型**定义发射/退休时序；**RTL** 实现契约。一个测试只有在四者一致时才算通过。

16.1 里程碑与验收门

门	产物	通过判据
M0: 数值参考	位精确的 MXINT8-B16、BF16、SFU、RMS、RoPE 库	全部 OCP 派生的边界向量通过；E 指数方程与 Q1.6 解释锁定；
M1: 模型质量	与 openpi 对齐的逐层与端到端评估	基线 RNE 通路满足约定的动作误差/rollout 成功率阈值； 实验性归一化捷径 (RMS_DEFER_EN) 被排除在外 ；给出 ceil vs floor 的端到端对比数据 (§ 4.4.4)；给出 18 层 BF16 残差链的累计偏置
M2: 周期模型	8 核事件模拟器 + HBM token bucket + 全局 pipe_en	
M3: 块级 RTL	MXU、DEQACC、VPU/SFU、RMS/Quant、存储器	在随机测试下与 M0/M2 达成周期与位一致

M4：子系统 UVM	1 个 PCore、GCore、CNET、CCU、MCU 集成	描述符、掩码、分体、停顿、ECC、abort 覆盖完备
M5：FPGA / 仿真加速	完整 8 核镜像	逐层比对与在支持的存储速率下的确定性周期数
M6：综合 / 功耗	目标工艺实现	频率、面积、IR/热、功耗区间重新基线化

附录 A 参数速查表

本表是 § 15.1 参数包的浓缩视图。**所有 RTL 常量必须由参数包派生，不得手写。**

类别	参数	值	派生关系
阵列	N_CORE	8	= query head 数
阵列	TILE	16	= TILE_R = TILE_C = MX 块 = B_kv
阵列	MXU_KIND	0（加法树）	1 = 脉动，仅 $\geq 32 \times 32$ 时考虑
模型	D_MODEL / D_HEAD / D_FFN	1024 / 256 / 4096	—
模型	N_LAYER / N_STEP	18 / 10	—
模型	PREFIX_CAP / SUFFIX_LEN	816 / 51	—
模型	ACT_HORIZON / ACT_DIM	50 / 32	—
派生	LOGICAL_SEQ	867	PREFIX_CAP + SUFFIX_LEN
派生	PHYS_SEQ	880	向上取整到 TILE 的倍数
派生	N_KV_BLOCK	55	PHYS_SEQ / TILE
派生	BLK_W	6	$\lceil \log_2(55) \rceil$
派生	SUFFIX_BLK0	51	PREFIX_CAP / TILE
数值	MX_E_BIAS / MX_FRAC_BITS	127 / 6	OCP 语义（D1）
数值	DOT_EXP_OFFSET	266	$2 * 127 + 2 * 6$
数值	QK_EXP_FOLD	−4	$1/\sqrt{256}$

数值	MX_ENC_MODE 默认	0 (ceil)	D2
时序	MXU_LAT / DEQACC_LAT	6 / 5	树结构
时序	PIPE_DRAIN	11	之和
位宽	DW_ST/DW_ACT/DW_SCALE	128	TILE*8
位宽	DW_PSUM/DW_VEC/ DW_CNET	512	TILE*32 (CNET 见 D9)
位宽	DW_RES / DW_XBC	256	TILE*16
存储	M_WORDS / OACC_WORDS / ZBUF_WORDS	3264 / 816 / 1632	—
存储	WEIGHT_TILE_B	272	TILE*TILE + TILE
存储	TILE_FIFO_ENT / TILE_AXI_BEATS	17 / 9	272/16、 $\lceil 272/32 \rceil$
存储	WFIFO_DEPTH / WFIFO_MIN_DEPTH	512 / 208	§ 10.4
存储	KVFIFO_DEPTH / XFIFO_DEPTH	128 / 128	§ 10.5

附录 B 描述符程序清单

下列符号清单是最小的层子程序。地址与 tile 计数由段表与参数实例化；花括号表示占用不相交资源、共享同一使能周期周期的并行操作。

代码块

```
1  LAYER_BEGIN ell, step
2      VPU  RMS_PASS_B    M=51 K=64                RESID -> XHAT                // 3,264 (暴露, D3)
3      { GEMM K_PROJ M=51 K=64 N=2 ; GEMM V_PROJ M=51 K=64 N=2 }                // 13,056
4      CNET_GATHER K_SUFFIX ; CNET_GATHER V_SUFFIX ; SYNC KV_SUFFIX_READY
5      GEMM Q_PROJ  M=51 K=64 N=16  POST=ROPE  -> Q_MEM                // 52,224
6      ATTN_INIT
7      QK blk=0 ; { QK blk=1 ; EXP blk=0 } ; { PV blk=0 ; EXP blk=1 }
8      LOOP b = 1..53:
9          { QK blk=b+1 ; VPU OACC_SCALE alpha[b] }                // α 完全隐藏
10         { PV blk=b    ; EXP blk=b+1 }
11         VPU  OACC_SCALE alpha[54]                // ★ 816 (暴露, D4)
12         { PV blk=54 ; A_FIN } ; SYNC O_READY
13         GEMM O_PROJ      M=51 K=128 N=8  POST=CNET_REDUCE_RESID        // 52,224
```

14	VPU	RMS_PASS_B	M=51	K=64	RESID -> XHAT	// 3,264 (暴露, D3)
15	GEMM	GATE_PROJ	M=51	K=64	N=32	
16	GEMM	UP_PROJ	M=51	K=64	N=32 POST=GELU_MUL -> ZBUF	// 合计
						208,896
17	GEMM	DOWN_PROJ	M=51	K=256	N=8 POST=CNET_REDUCE_RESID	// 104,448
18	LAYER_END					

程序检查	期望
Attention 块计数	55 个 QK、55 个 EXP、55 个 PV、54 个非平凡 α ; α[54] 暴露
归一化事件	每层两次，加层 17 之后的一次最终归一化（共 37 次/步）
CNET 残差归约	O 投影与 down 投影，各一次
CNET gather	suffix K 与 suffix V，各一次
最终次序	final RMS $\rightarrow$ action_out_proj $\rightarrow$ Euler 更新 $\rightarrow$ BF16 存储
掩码模式	仅 blk $\geq$ 51 的 QK 置 MASK_MODE=1